Advanced Representation Learning
(고급표현학습)

Instructor: Donghyeon Cho

The Department of Computer Science

Hanyang University

한양대학교
HANYANG UNIVERSITY

Contents of Table

- 1. Learning from Memory Bank
  - MoCo
  - CURL
- 2. Clustering
  - DeepCluster
  - SwAV
- 3. Working with Supervised Datasets
  - CLIP
  - Supervised Contrastive Learning

Learning from Memory Bank

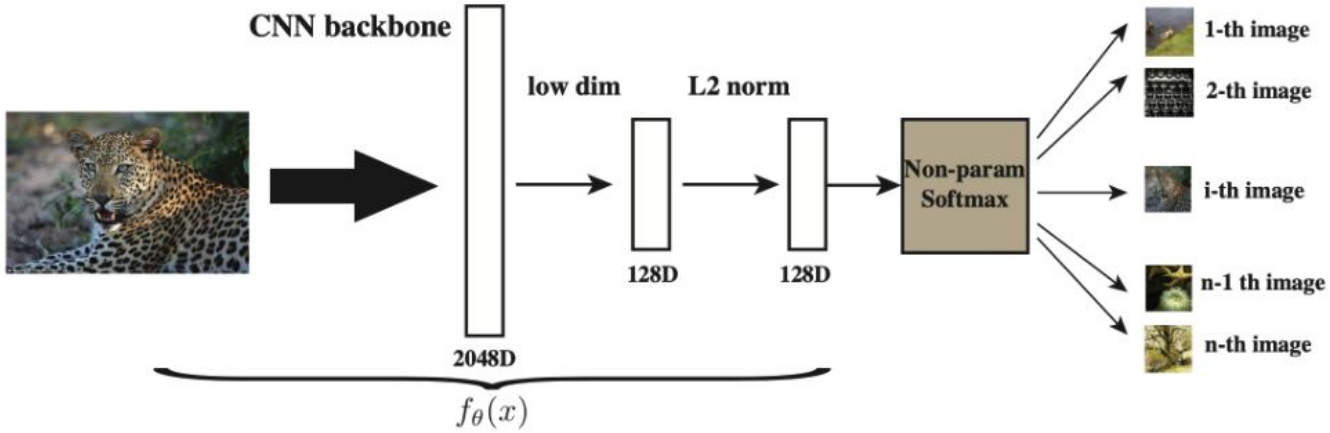
Memory Bank

- **Large numbers of negative samples** improve contrastive learning.
- Computing embeddings for all negatives is computationally expensive.
- **Store** previously computed embeddings in memory.
- **Reuse** stored embeddings as negative samples during training.
- **Key Idea**: Trade stale representations for efficient large-scale contrastive learning.

Mini-batch Negatives → Limited but Fresh
Memory Bank Negatives → Large but Slightly Stale

Instance Discrimination with Memory Bank

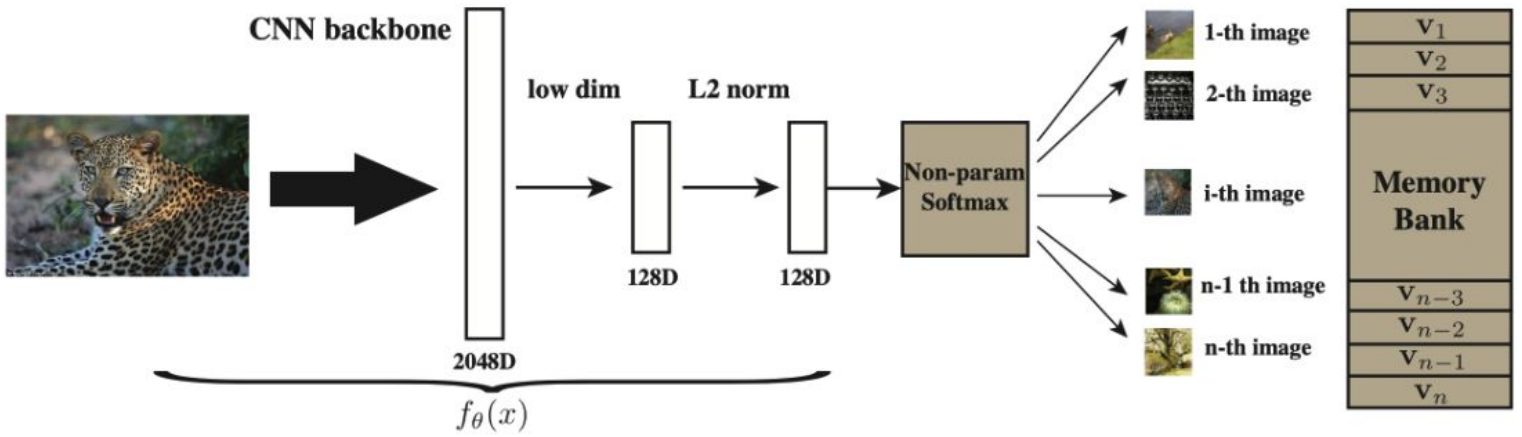
- **Instance contrastive learning**
  - **Each** image is considered a **unique class**.
  - Positive pair: the same image.
  - Negative pairs: all other images.
  - → Unlike conventional classification, instance discrimination treats every image as a unique class and learns to distinguish it from all other images.



Wu et al., CVPR 2018, Unsupervised Feature Learning via Non-Parametric Instance-level Discrimination

Instance Discrimination with Memory Bank

- **Instance contrastive learning**
  - **Unfeasible** to train a softmax layer with these many heads.
  - → Instead of calculating all classes, approximated by using **Noise Contrastive Estimation (NCE)**, which samples only some negatives.
  - **Memory bank** enables efficient large-scale instance discrimination.
    - Storing sample representation in the database from past iterations.



Wu et al., CVPR 2018, Unsupervised Feature Learning via Non-Parametric Instance-level Discrimination

Instance Discrimination with Memory Bank

- Non-parametric classifier predicts the probability of a sample *v* belonging to class *i* :

embedding

$$\mathbf{v} = f_\theta(x)$$

Probability

$$P(C = i|\mathbf{v}) = \frac{\exp(\mathbf{v}_i^\top \mathbf{v}/\tau)}{\sum_{j=1}^{n} \exp(\mathbf{v}_j^\top \mathbf{v}/\tau)}$$

- **Memory Bank** for storing sample representation in the database from past iterations.

Memory bank

$$V = \{\mathbf{v}_i\}$$

Wu et al., CVPR 2018, Unsupervised Feature Learning via Non-Parametric Instance-level Discrimination

Instance Discrimination with Memory Bank

- Access to the representations of all the samples
  - too expensive in practice → Monte Carlo approximation using a random subset of M

$$P(i|\mathbf{v}) = \frac{\exp(\mathbf{v}^\top \mathbf{f}_i/\tau)}{\sum_{j=1}^{N} \exp(\mathbf{v}_j^\top \mathbf{f}_i/\tau)} \simeq \frac{\exp(\mathbf{v}^\top \mathbf{f}_i/\tau)}{\frac{N}{M}\sum_{k=1}^{M} \exp(\mathbf{v}_{j_k}^\top \mathbf{f}_i/\tau)}$$

- Introduced an extra term for positive samples in the loss function based on the proximal optimization method.

$$\mathcal{L}_{\text{instance}} = -\mathbb{E}_{P_d}\left[\log h(i, \mathbf{v}_i^{(t-1)}) - \lambda \|\mathbf{v}_i^{(t)} - \mathbf{v}_i^{(t-1)}\|_2^2\right] - M\mathbb{E}_{P_n}\left[\log(1 - h(i, \mathbf{v}'^{(t-1)})\right]$$

$$h(i, \mathbf{v}) = \frac{P(i|\mathbf{v})}{P(i|\mathbf{v}) + MP_n(i)} \quad \text{where the noise distribution is uniform } P_n = 1/N$$

Given feature v and ID i, what is the probability that it originated from the actual data source?

gradually vanish as the learned embedding converges

Wu et al., CVPR 2018, Unsupervised Feature Learning via Non-Parametric Instance-level Discrimination

Instance Discrimination with Memory Bank

- Memory Bank: Pros and Cons
- Pros
  - Large number of negative samples
  - Low computational cost
- Cons
  - Stored embeddings become stale
  - Current queries are compared with outdated features
- Question: Can we keep a large dictionary while reducing feature staleness?

Wu et al., CVPR 2018, Unsupervised Feature Learning via Non-Parametric Instance-level Discrimination

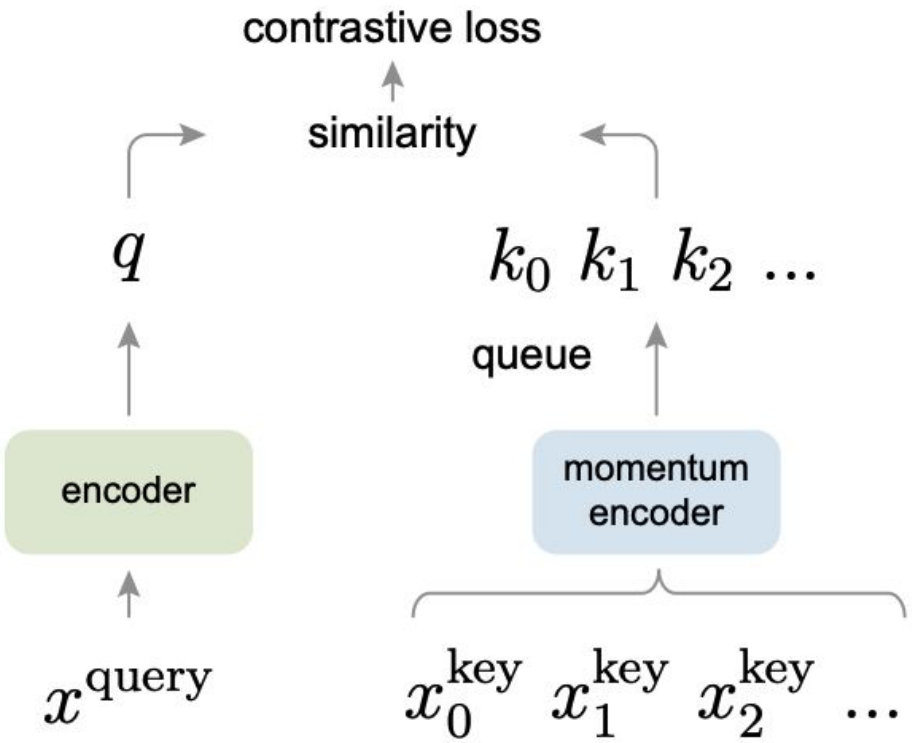
MoCo

- **Momentum Contrast (MoCo)**
  - Maintains a dynamic dictionary of negative keys.
  - Stores recent representations in a **FIFO queue.**
  - Encodes keys using a **momentum encoder.**
  - Enables a large and consistent dictionary for contrastive learning.
- **Key Idea**: Replace the memory bank with a momentum-updated queue.

He et al., CVPR 2020, Momentum Contrast for Unsupervised Visual Representation Learning

MoCo

- **Momentum Contrast (MoCo)**



$$\mathcal{L}_{\text{MoCo}} = -\log \frac{\exp(\mathbf{q} \cdot \mathbf{k}^+/\tau)}{\sum_{i=1}^{N} \exp(\mathbf{q} \cdot \mathbf{k}_i/\tau)}$$

k+ is differently augmented query

Key differences to SimCLR:

- Keep a running **queue** of keys (negative samples).
- Compute gradients and update the encoder **only through the queries.**
- Decouple min-batch size with the number of keys: can support **a large number of negative samples.**
- The key encoder is **slowly progressing** through the momentum update rules:

$$\theta_k \leftarrow m\theta_k + (1-m)\theta_q$$

He et al., CVPR 2020, Momentum Contrast for Unsupervised Visual Representation Learning

MoCo

- Codes.

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
   x_q = aug(x) # a randomly augmented version
   x_k = aug(x) # another randomly augmented version

   q = f_q.forward(x_q) # queries: NxC
   k = f_k.forward(x_k) # keys: NxC
   k = k.detach() # no gradient to keys

   # positive logits: Nx1
   l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

   # negative logits: NxK
   l_neg = mm(q.view(N,C), queue.view(C,K))

   # logits: Nx(1+K)
   logits = cat([l_pos, l_neg], dim=1)

   # contrastive loss, Eqn.(1)
   labels = zeros(N) # positives are the 0-th
   loss = CrossEntropyLoss(logits/t, labels)

   # SGD update: query network
   loss.backward()
   update(f_q.params)

   # momentum update: key network
   f_k.params = m*f_k.params+(1-m)*f_q.params

   # update dictionary
   enqueue(queue, k) # enqueue the current minibatch
   dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

Generate a positive pair by sampling data augmentation functions

No gradient through the key

Use the running queue of keys as the negative samples

InfoNCE loss

Update f_k through momentum

Update the FIFO negative sample queue

He et al., CVPR 2020, Momentum Contrast for Unsupervised Visual Representation Learning

MoCo

- Comparisons.

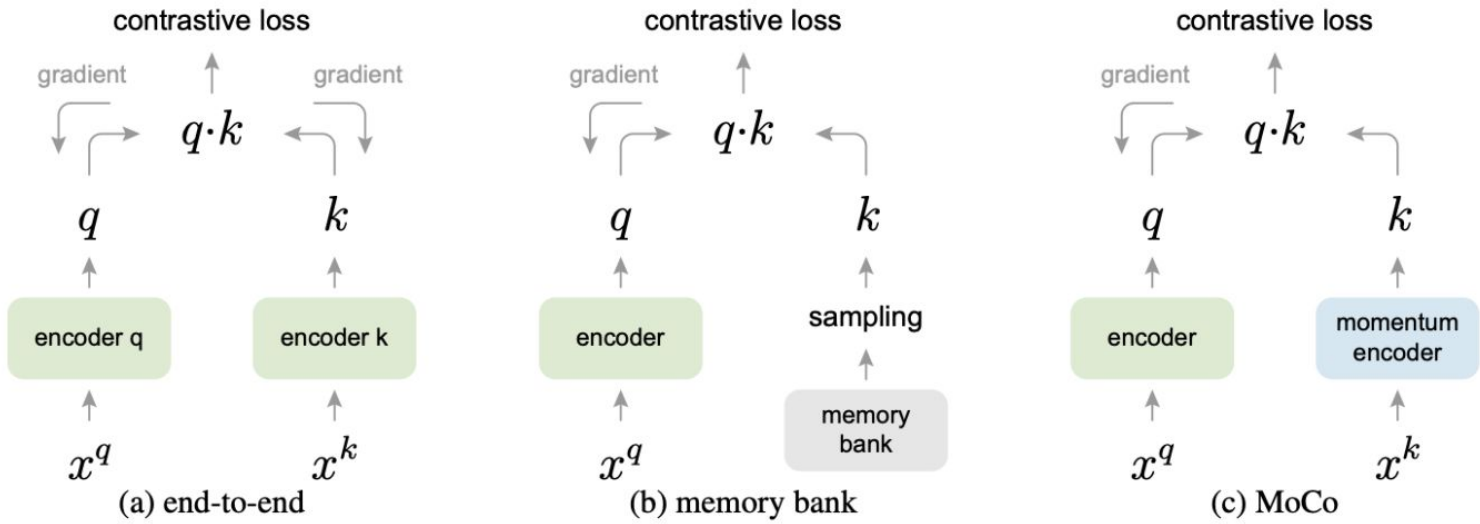


Figure 2. **Conceptual comparison of three contrastive loss mechanisms** (empirical comparisons are in Figure 3 and Table 3). Here we illustrate one pair of query and key. The three mechanisms differ in how the keys are maintained and how the key encoder is updated. **(a)**: The encoders for computing the query and key representations are updated ***end-to-end*** by back-propagation (the two encoders can be different). **(b)**: The key representations are sampled from a ***memory bank*** [61]. **(c)**: ***MoCo*** encodes the new keys on-the-fly by a momentum-updated encoder, and maintains a queue (not illustrated in this figure) of keys.

He et al., CVPR 2020, Momentum Contrast for Unsupervised Visual Representation Learning

MoCo

- Comparisons.

	End-to-End	Memory Bank	MoCo
Query	Current encoder	Current encoder	Current encoder
Key	Current encoder	Memory bank	Momentum encoder
Negative Source	Mini-batch	Memory bank	FIFO queue
Dictionary Size	Small	Large	Large
Feature Freshness	Fresh	Stale	Consistent
Encoder Update	Backpropagation	Backpropagation	Momentum Update

MoCo is essentially a hybrid of end-to-end learning and memory banks: it keeps a large dictionary like a memory bank while generating more consistent keys using a momentum encoder.

He et al., CVPR 2020, Momentum Contrast for Unsupervised Visual Representation Learning

MoCo

- Comparisons.

method	architecture	#params (M)	accuracy (%)
Exemplar [17]	R50w3×	211	46.0 [38]
RelativePosition [13]	R50w2×	94	51.4 [38]
Jigsaw [45]	R50w2×	94	44.6 [38]
Rotation [19]	Rv50w4×	86	55.4 [38]
Colorization [64]	R101*	28	39.6 [14]
DeepCluster [3]	VGG [53]	15	48.4 [4]
BigBiGAN [16]	R50	24	56.6
	Rv50w4×	86	61.3
methods based on contrastive learning follow:			
InstDisc [61]	R50	24	54.0
LocalAgg [66]	R50	24	58.8
CPC v1 [46]	R101*	28	48.7
CPC v2 [35]	$R170^*_{wider}$	303	65.9
CMC [56]	$R50_{L+ab}$	47	64.1†
	$R50w2\times_{L+ab}$	188	68.4†
AMDIM [2]	$AMDIM_{small}$	194	63.5†
	$AMDIM_{large}$	626	68.1†
MoCo	R50	24	60.6
	RX50	46	63.9
	R50w2×	94	65.4
	R50w4×	375	**68.6**

Table 1. **Comparison under the linear classification protocol on ImageNet**. The figure visualizes the table. All are reported as unsupervised pre-training on the ImageNet-1M training set, followed by supervised linear classification trained on frozen features, evaluated on the validation set. The parameter counts are those of the feature extractors. We compare with improved re-implementations if available (referenced after the numbers).

pre-train	AP_{50}	AP	AP_{75}
random init.	64.4	37.9	38.6
super. IN-1M	81.4	54.0	59.1
MoCo IN-1M	81.1 (−0.3)	54.6 (**+0.6**)	59.9 (**+0.8**)
MoCo IG-1B	81.6 (+0.2)	55.5 (**+1.5**)	61.2 (**+2.1**)

(a) Faster R-CNN, R50-**dilated-C5**

pre-train	AP_{50}	AP	AP_{75}
random init.	60.2	33.8	33.1
super. IN-1M	81.3	53.5	58.8
MoCo IN-1M	81.5 (+0.2)	55.9 (**+2.4**)	62.6 (**+3.8**)
MoCo IG-1B	82.2 (**+0.9**)	57.2 (**+3.7**)	63.7 (**+4.9**)

(b) Faster R-CNN, R50-**C4**

Table 2. **Object detection fine-tuned on PASCAL VOC** `trainval07+12`. Evaluation is on `test2007`: AP_{50} (default VOC metric), AP (COCO-style), and AP_{75}, averaged over 5 trials. All are fine-tuned for 24k iterations (~23 epochs). In the brackets are the gaps to the ImageNet supervised pre-training counterpart. In green are the gaps of at least **+0.5** point.

	R50-dilated-C5			R50-C4		
pre-train	AP_{50}	AP	AP_{75}	AP_{50}	AP	AP_{75}
end-to-end	79.2	52.0	56.6	80.4	54.6	60.3
memory bank	79.8	52.9	57.9	80.6	54.9	60.6
MoCo	**81.1**	**54.6**	**59.9**	**81.5**	**55.9**	**62.6**

Table 3. **Comparison of three contrastive loss mechanisms** on PASCAL VOC object detection, fine-tuned on `trainval07+12` and evaluated on `test2007` (averages over 5 trials). All models are implemented by us (Figure 3), pre-trained on IN-1M, and fine-tuned using the same settings as in Table 2.

He et al., CVPR 2020, Momentum Contrast for Unsupervised Visual Representation Learning

MoCo-V2

- **MoCo v2 (2020)**
  - Adopts the **MLP projection head** from SimCLR.
  - Uses **stronger data augmentation** for contrastive learning.
  - Improves representation quality and transfer performance.
  - Retains the efficient momentum encoder and queue mechanism.
- **Key Idea**: Combine MoCo with SimCLR's effective training strategies.

Chen et al., 2020, Improved Baselines with Momentum Contrastive Learning

MoCo-V2

- **MoCo v2 (2020)**

case	unsup. pre-train MLP	aug+	cos	epochs	ImageNet acc.	VOC detection AP_{50}	AP	AP_{75}
supervised					76.5	81.3	53.5	58.8
MoCo v1				200	60.6	81.5	55.9	62.6
(a)	✓			200	66.2	82.0	56.4	62.6
(b)		✓		200	63.4	82.2	56.8	63.2
(c)	✓	✓		200	67.3	**82.5**	57.2	63.9
(d)	✓	✓	✓	200	67.5	82.4	57.0	63.6
(e)	✓	✓	✓	**800**	**71.1**	**82.5**	**57.4**	**64.0**

Table 1. **Ablation of MoCo baselines**, evaluated by ResNet-50 for (i) ImageNet linear classification, and (ii) fine-tuning VOC object detection (mean of 5 trials). "**MLP**": with an MLP head; "**aug+**": with extra blur augmentation; "**cos**": cosine learning rate schedule.

Key takeaways:

- Non-linear projection head and strong data augmentation are crucial for contrastive learning.

Chen et al., 2020, Improved Baselines with Momentum Contrastive Learning

MoCo-V2

- **MoCo v2 (2020)**

case	unsup. pre-train MLP	aug+	cos	epochs	batch	ImageNet acc.
MoCo v1 [6]				200	256	60.6
SimCLR [2]	✓	✓	✓	200	256	61.9
SimCLR [2]	✓	✓	✓	200	8192	66.6
MoCo v2	✓	✓	✓	200	256	**67.5**
*results of **longer** unsupervised training follow:*						
SimCLR [2]	✓	✓	✓	1000	4096	69.3
MoCo v2	✓	✓	✓	800	256	**71.1**

Table 2. **MoCo *vs.* SimCLR**: ImageNet linear classifier accuracy (**ResNet-50, 1-crop 224×224**), trained on features from unsupervised pre-training. "aug+" in SimCLR includes blur and stronger color distortion. SimCLR ablations are from Fig. 9 in [2] (we thank the authors for providing the numerical results).

Key takeaways:

- Non-linear projection head and strong data augmentation are crucial for contrastive learning.
- Decoupling mini-batch size with negative sample size allows MoCo-V2 to outperform SimCLR with smaller batch size (256 vs. 8192).

Chen et al., 2020, Improved Baselines with Momentum Contrastive Learning

MoCo-V2

- **MoCo v2 (2020)**

mechanism	batch	memory / GPU	time / 200-ep.
MoCo	256	**5.0G**	**53 hrs**
end-to-end	256	7.4G	65 hrs
end-to-end	4096	93.0G†	n/a

Table 3. **Memory and time cost** in 8 V100 16G GPUs, implemented in PyTorch. †: based on our estimation.

Key takeaways:

- Non-linear projection head and strong data augmentation are crucial for contrastive learning.
- Decoupling mini-batch size with negative sample size allows MoCo-V2 to outperform SimCLR with smaller batch size (256 vs. 8192).
- All with much smaller memory footprint! ("end-to-end" means SimCLR here)

Chen et al., 2020, Improved Baselines with Momentum Contrastive Learning

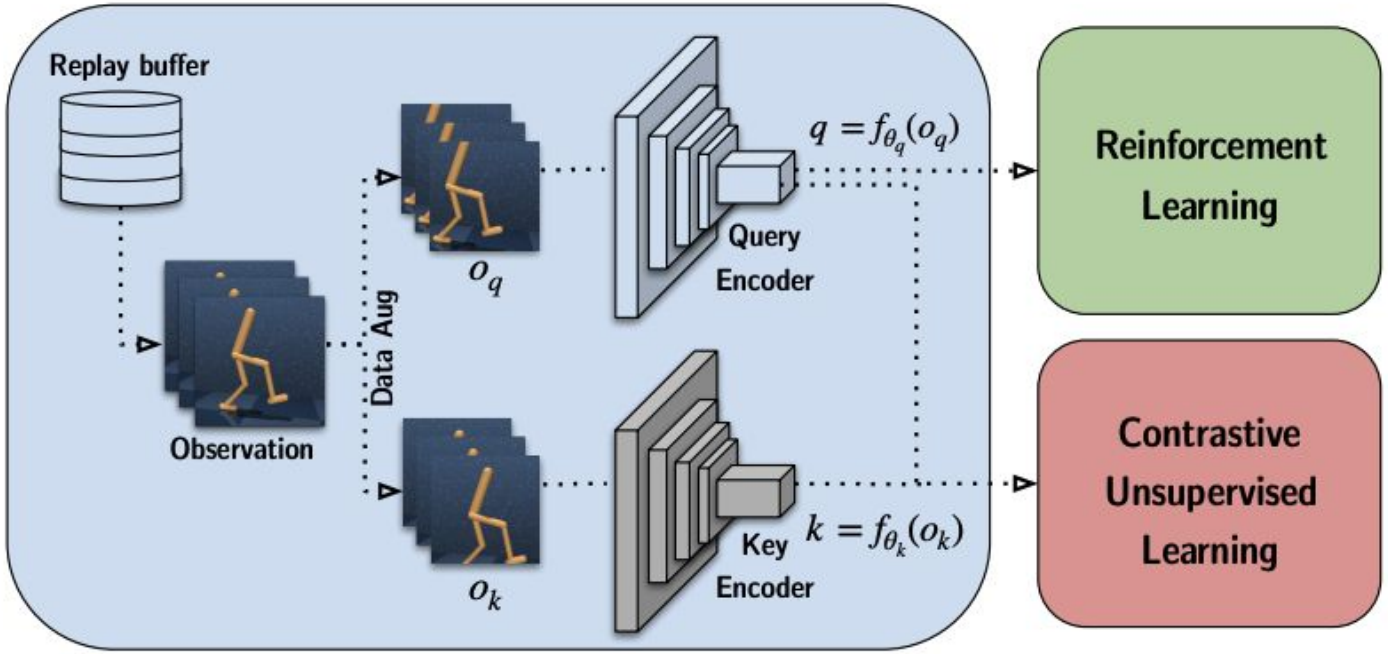
CURL

- **CURL (2020)**
  - Applies contrastive learning to **reinforcement learning**.
  - Learns visual representations from **raw** observations.
  - Uses random crop **augmentation** to create positive pairs.
  - Employs a momentum encoder and contrastive loss, following **MoCo**.
- **Key Idea**: Improve sample-efficient reinforcement learning through contrastive representation learning.

Srinivas et al., 2020, CURL: Contrastive Unsupervised Representations for Reinforcement Learning

CURL

- **CURL (2020)**



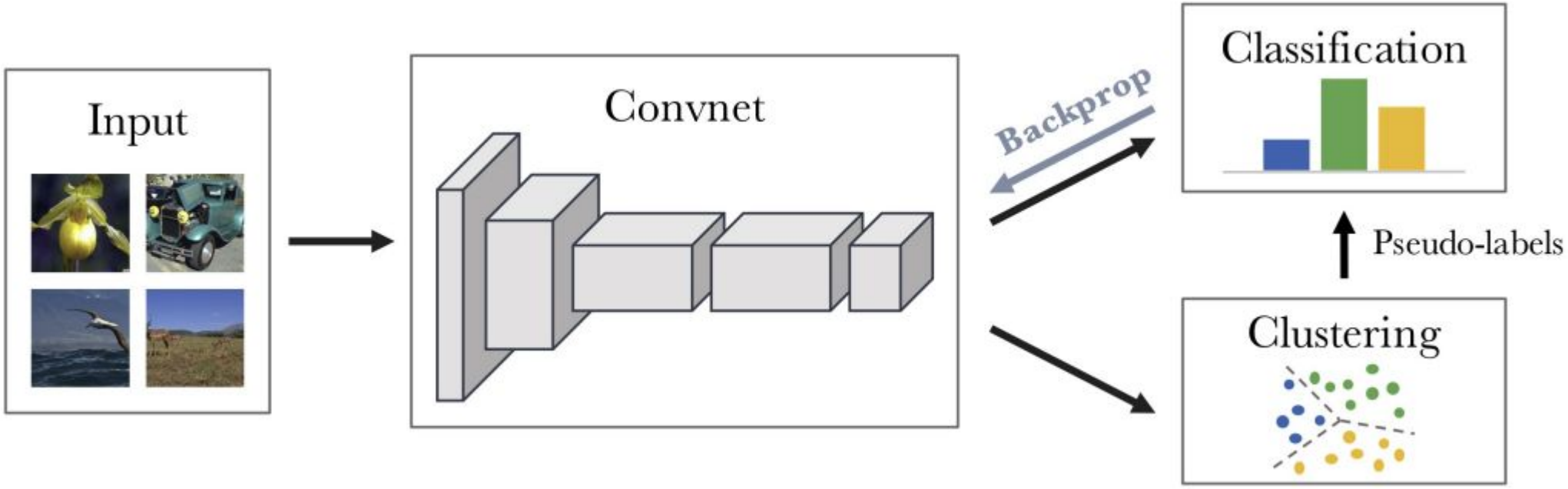
- Sample observations from the replay buffer.
- Generate two augmented views of each observation.
- Encode them using the query encoder and momentum key encoder.
- Train the encoder with contrastive learning.
- Simultaneously optimize the RL policy/value network using the learned representation.

Srinivas et al., 2020, CURL: Contrastive Unsupervised Representations for Reinforcement Learning

Clustering

DeepCluster

- Unlike contrastive learning, it does not create positive/negative pairs, but uses clustering results as pseudo labels.



Caron et al., 2018, Deep Clustering for Unsupervised Learning of Visual Features

DeepCluster

- **DeepCluster (2018)**
  - Cluster image features using the current representation.
  - Treat cluster assignments as pseudo labels.
  - Train the network to predict the assigned clusters.
  - Repeat clustering and representation learning iteratively.
- **Key Idea**: Alternate between clustering and classification.

Process
- Extract image representations.
- Cluster the representations using k-means.
- Assign pseudo labels from cluster IDs.
- Update the network using the pseudo labels.
- Repeat until convergence.

Caron et al., 2018, Deep Clustering for Unsupervised Learning of Visual Features

DeepCluster

- **DeepCluster (2018)**

Method	ImageNet					Places				
	conv1	conv2	conv3	conv4	conv5	conv1	conv2	conv3	conv4	conv5
Places labels	–	–	–	–	–	22.1	35.1	40.2	43.3	44.6
ImageNet labels	19.3	36.3	44.2	48.3	50.5	22.7	34.8	38.4	39.4	38.7
Random	11.6	17.1	16.9	16.3	14.1	15.7	20.3	19.8	19.1	17.5
Pathak *et al.* [38]	14.1	20.7	21.0	19.8	15.5	18.2	23.2	23.4	21.9	18.4
Doersch *et al.* [25]	16.2	23.3	30.2	31.7	29.6	19.7	26.7	31.9	32.7	30.9
Zhang *et al.* [28]	12.5	24.5	30.4	31.5	30.3	16.0	25.7	29.6	30.3	29.7
Donahue *et al.* [20]	17.7	24.5	31.0	29.9	28.0	21.4	26.2	27.1	26.1	24.0
Noroozi and Favaro [26]	**18.2**	28.8	34.0	33.9	27.1	23.0	32.1	35.5	34.8	31.3
Noroozi *et al.* [45]	18.0	**30.6**	34.3	32.5	25.7	**23.3**	**33.9**	36.3	34.7	29.6
Zhang *et al.* [43]	17.7	29.3	35.4	35.2	32.8	21.3	30.7	34.0	34.1	32.5
DeepCluster	12.9	29.2	**38.2**	**39.8**	**36.1**	18.6	30.8	**37.0**	**37.5**	**33.1**

Table 1: Linear classification on ImageNet and Places using activations from the convolutional layers of an AlexNet as features. We report classification accuracy on the central crop. Numbers for other methods are from Zhang *et al.* [43].

Method	Classification		Detection		Segmentation	
	FC6-8	ALL	FC6-8	ALL	FC6-8	ALL
ImageNet labels	78.9	79.9	–	56.8	–	48.0
Random-rgb	33.2	57.0	22.2	44.5	15.2	30.1
Random-sobel	29.0	61.9	18.9	47.9	13.0	32.0
Pathak *et al.* [38]	34.6	56.5	–	44.5	–	29.7
Donahue *et al.* [20]*	52.3	60.1	–	46.9	–	35.2
Pathak *et al.* [27]	–	61.0	–	52.2	–	–
Owens *et al.* [44]*	52.3	61.3	–	–	–	–
Wang and Gupta [29]*	55.6	63.1	32.8†	47.2	26.0†	35.4†
Doersch *et al.* [25]*	55.1	65.3	–	51.1	–	–
Bojanowski and Joulin [19]*	56.7	65.3	33.7†	49.4	26.7†	37.1†
Zhang *et al.* [28]*	61.5	65.9	43.4†	46.9	35.8†	35.6
Zhang *et al.* [43]*	63.0	67.1	–	46.7	–	36.0
Noroozi and Favaro [26]	–	67.6	–	53.2	–	37.6
Noroozi *et al.* [45]	–	67.7	–	51.4	–	36.6
DeepCluster	**70.4**	**73.7**	**51.4**	**55.4**	**43.2**	**45.1**

Table 2: Comparison of the proposed approach to state-of-the-art unsupervised feature learning on classification, detection and segmentation on PASCAL VOC. * indicates the use of the data-dependent initialization of Krähenbühl *et al.* [68]. Numbers for other methods produced by us are marked with a †.

Contributions

- Introduced clustering-based self-supervised learning.
- Established pseudo-label learning as an alternative to contrastive learning.
- Achieved state-of-the-art performance among unsupervised methods at the time.
- Inspired subsequent clustering-based methods such as SwAV and DINO.

Caron et al., 2018, Deep Clustering for Unsupervised Learning of Visual Features

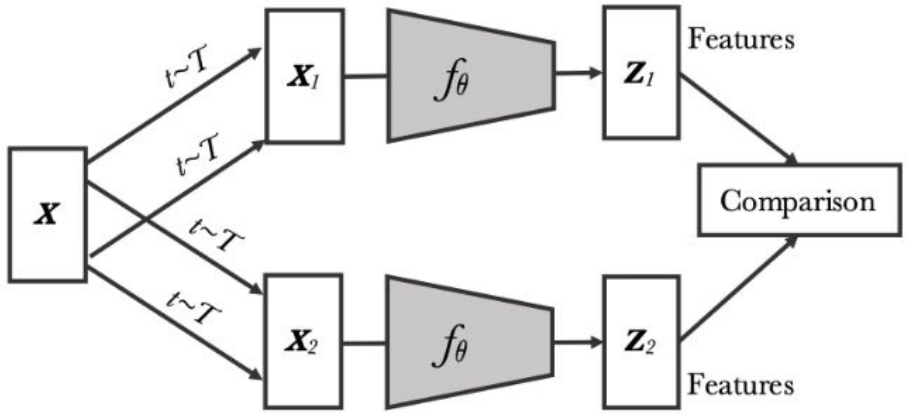
SwAV

- SwAV (Swapping Assignments between multiple Views)
  - Learn visual representations by predicting cluster assignments.
  - Compute **online** cluster assignments for augmented views.
  - Predict the assignment of one view from another view.
  - **Eliminate the need for explicit pairwise comparisons or offline clustering.**
- **Key Idea**: Learn by swapping cluster assignments between multiple views.

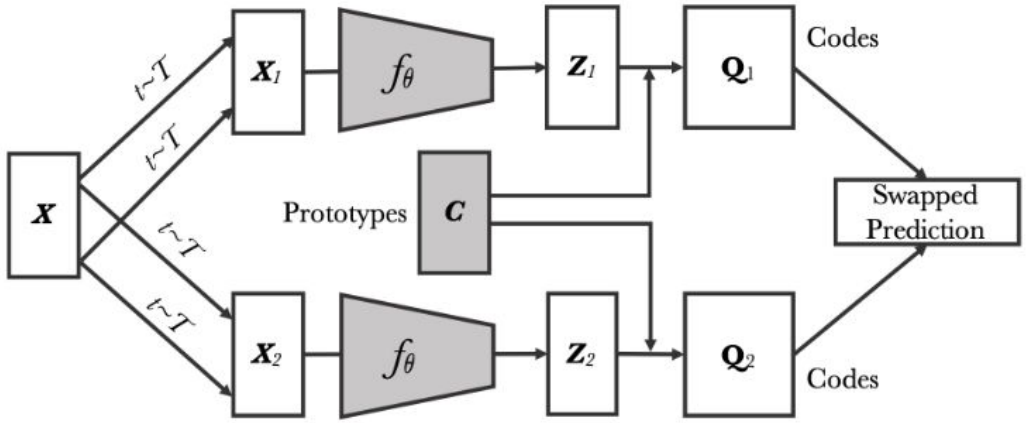
Caron et al., NeurIPS 2020, Unsupervised Learning of Visual Features by Contrasting Cluster Assignments

SwAV

- SwAV (Swapping Assignments between multiple Views)
  - Given features of two augmented images z, computes codes q, measuring swapped loss.



Contrastive instance learning



Swapping Assignments between Views (Ours)

$$\mathcal{L}_{\mathrm{SwAV}}(\mathbf{z}_t, \mathbf{z}_s) = \ell(\mathbf{z}_t, \mathbf{q}_s) + \ell(\mathbf{z}_s, \mathbf{q}_t)$$

$$\ell(\mathbf{z}_t, \mathbf{q}_s) = -\sum_k \mathbf{q}_s^{(k)} \log \mathbf{p}_t^{(k)} \text{ where } \mathbf{p}_t^{(k)} = \frac{\exp(\mathbf{z}_t^\top \mathbf{c}_k / \tau)}{\sum_{k'} \exp(\mathbf{z}_t^\top \mathbf{c}_{k'} / \tau)}$$

cross entropy between the predicted code and a set of K trainable prototype vectors C

Caron et al., NeurIPS 2020, Unsupervised Learning of Visual Features by Contrasting Cluster Assignments

SwAV

- SwAV (Swapping Assignments between multiple Views)

Method	Arch.	Param.	Top1
Supervised	R50	24	76.5
Colorization [65]	R50	24	39.6
Jigsaw [46]	R50	24	45.7
NPID [58]	R50	24	54.0
BigBiGAN [15]	R50	24	56.6
LA [68]	R50	24	58.8
NPID++ [44]	R50	24	59.0
MoCo [24]	R50	24	60.6
SeLa [2]	R50	24	61.5
PIRL [44]	R50	24	63.6
CPC v2 [28]	R50	24	63.8
PCL [37]	R50	24	65.9
SimCLR [10]	R50	24	70.0
MoCov2 [11]	R50	24	71.1
SwAV	R50	24	**75.3**

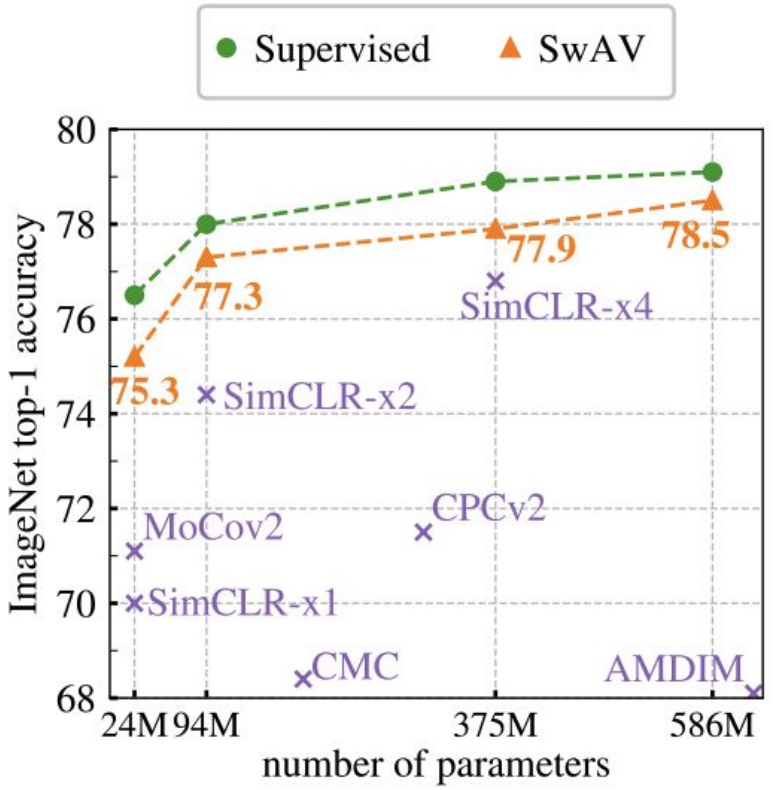


Figure 2: **Linear classification on ImageNet.** Top-1 accuracy for linear models trained on frozen features from different self-supervised methods. **(left)** Performance with a standard ResNet-50. **(right)** Performance as we multiply the width of a ResNet-50 by a factor $\times 2$, $\times 4$, and $\times 5$.

Caron et al., NeurIPS 2020, Unsupervised Learning of Visual Features by Contrasting Cluster Assignments

SwAV

- SwAV (Swapping Assignments between multiple Views)

Table 2: **Transfer learning on downstream tasks.** Comparison between features from ResNet-50 trained on ImageNet with SwAV or supervised learning. We consider two settings. (1) Linear classification on top of frozen features. We report top-1 accuracy on all datasets except VOC07 where we report mAP. (2) Object detection with finetuned features on VOC07+12 `trainval` using Faster R-CNN [50] and on COCO [38] using Mask R-CNN [26] or DETR [6]. We report the most standard detection metrics for these datasets: AP_{50} on VOC07+12 and AP on COCO.

	Linear Classification			Object Detection		
	Places205	VOC07	iNat18	VOC07+12 (Faster R-CNN R50-C4)	COCO (Mask R-CNN R50-FPN)	COCO (DETR)
Supervised	53.2	87.5	46.7	81.3	39.7	40.8
SwAV	**56.7**	**88.9**	**48.6**	**82.6**	**41.6**	**42.1**

Caron et al., NeurIPS 2020, Unsupervised Learning of Visual Features by Contrasting Cluster Assignments

SwAV

- SwAV (Swapping Assignments between multiple Views)
  - SwAV replaces feature matching with prototype assignment prediction, learning representations by predicting the cluster assignment of one view from another without explicit negative pairs.

Contrastive Learning	SwAV
Feature ↔ Feature	Feature ↔ Prototype
Positive/Negative pairs	Prototype assignments
Similarity matching	Swapped assignment prediction
Pairwise comparison	Online clustering

Caron et al., NeurIPS 2020, Unsupervised Learning of Visual Features by Contrasting Cluster Assignments

Working with Supervised Datasets

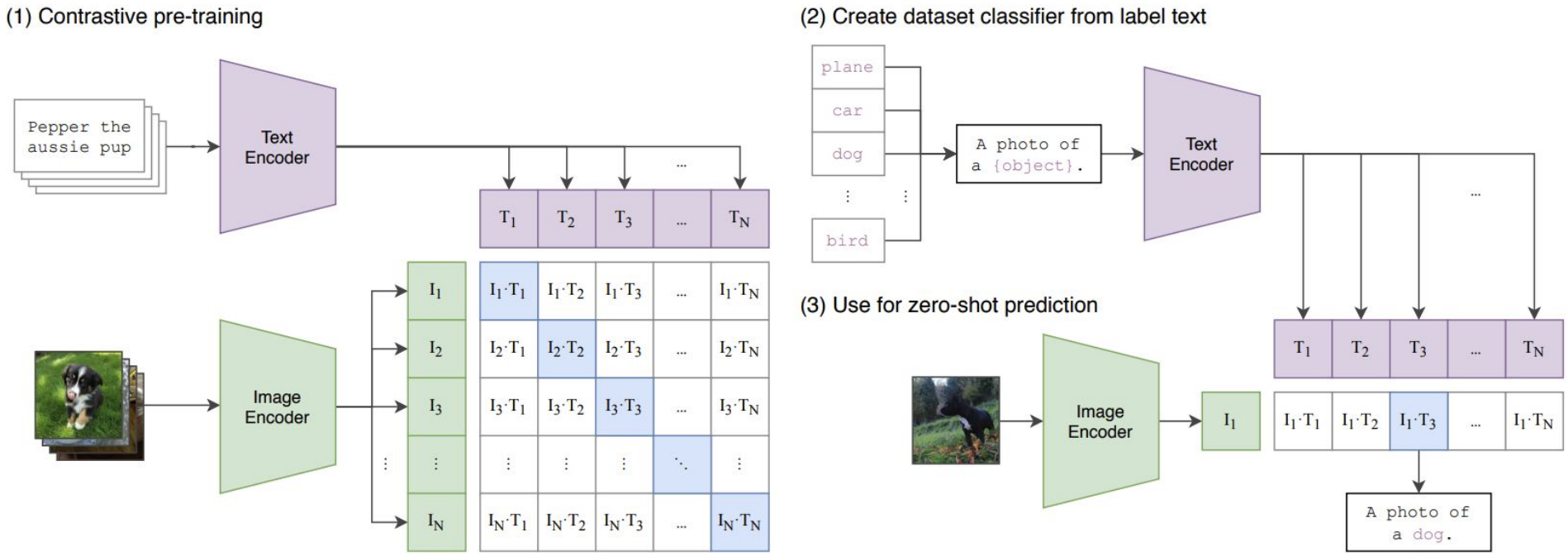
CLIP

- CLIP (Contrastive Language-Image Pre-training)
  - Jointly learns **image and text** representations.
  - **Contrastively** matches images with their corresponding captions.
  - **Aligns** visual and language embeddings in a **shared space**.
  - Enables zero-shot image recognition using natural language prompts.
- **Key Idea**: Learn visual representations by aligning images and text.

Radford et al., PMLR 2021, Learning Transferable Visual Models From Natural Language Supervision

CLIP

- CLIP (Contrastive Language-Image Pre-training)



CLIP Training

- Encode images and captions into a shared embedding space.
- Compute an $N \times N$ image-text similarity matrix.
- Maximize similarity for matched pairs.
- Minimize similarity for mismatched pairs using a **symmetric contrastive loss**.

Key Idea: Learn aligned image and text representations.

CLIP Inference

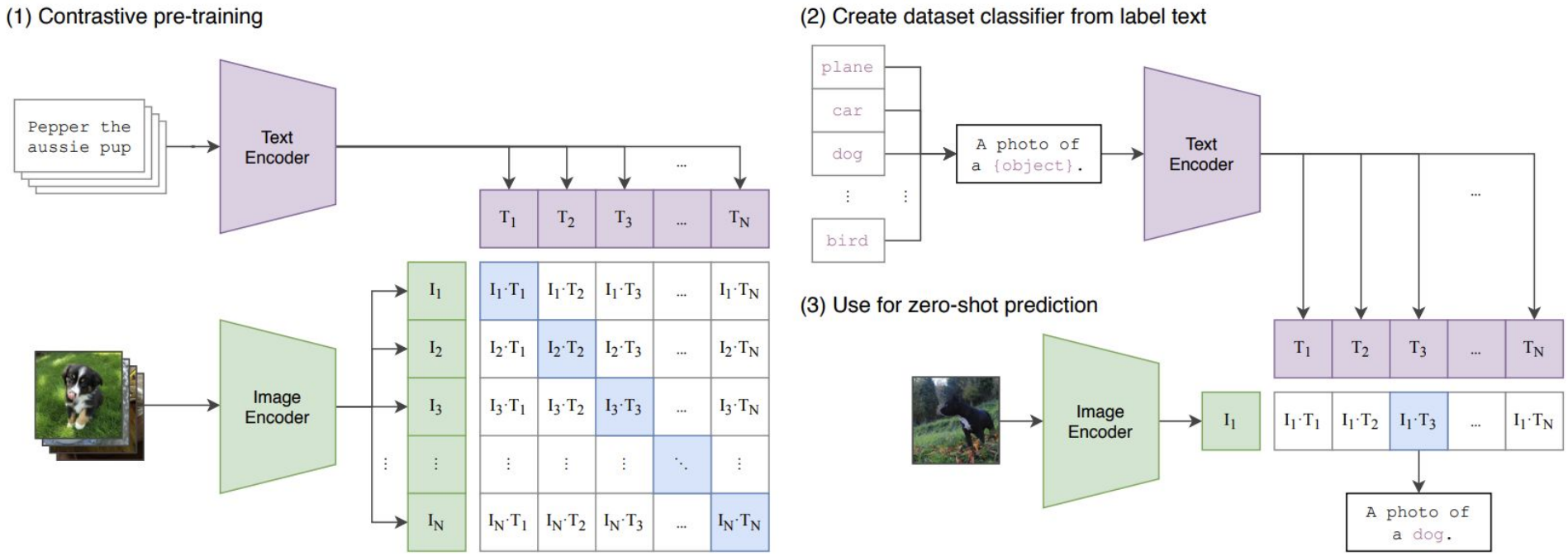
- Encode an input image.
- Encode candidate text prompts.
- Compute image-text similarities.
- Predict the text prompt with the highest similarity.

Key Idea: Classification is performed by image-text matching instead of a trained classifier.

Radford et al., PMLR 2021, Learning Transferable Visual Models From Natural Language Supervision

CLIP

- CLIP (Contrastive Language-Image Pre-training)



CLIP Training

- Encode images and captions into a shared embedding space.
- Compute an $N \times N$ image-text similarity matrix.
- Maximize similarity for matched pairs.
- Minimize similarity for mismatched pairs using a **symmetric contrastive loss**.

Key Idea: Learn aligned image and text representations.

CLIP Inference

- Encode an input image.
- Encode candidate text prompts.
- Compute image-text similarities.
- Predict the text prompt with the highest similarity.

Key Idea: Classification is performed by image-text matching instead of a trained classifier.

Radford et al., PMLR 2021, Learning Transferable Visual Models From Natural Language Supervision

CLIP

- CLIP (Contrastive Language-Image Pre-training)

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

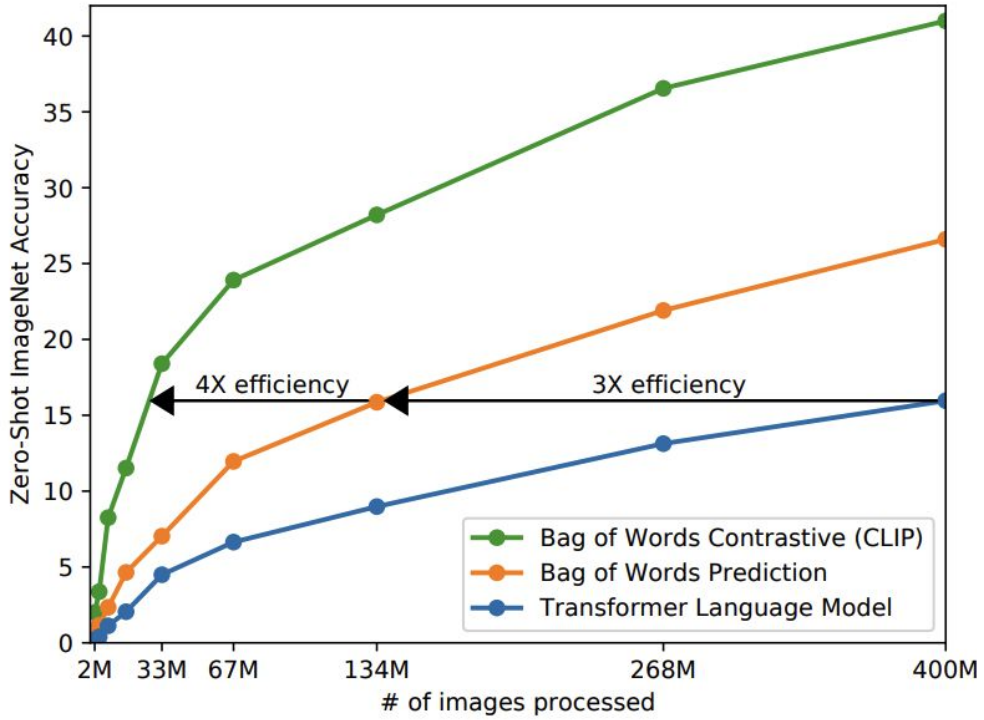
# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Radford et al., PMLR 2021, Learning Transferable Visual Models From Natural Language Supervision

CLIP

- CLIP (Contrastive Language-Image Pre-training)



The success of CLIP comes less from sophisticated language modeling and more from **contrastive alignment between images and natural language.**

Key Findings

- **Contrastive learning** is approximately **4× more data-efficient** than caption prediction.
- A simple **Bag-of-Words (BoW)** text encoder outperforms a Transformer language model.
- Rich language supervision is more important than complex text modeling.
- Contrastive alignment enables efficient large-scale vision-language pretraining.

Radford et al., PMLR 2021, Learning Transferable Visual Models From Natural Language Supervision

CLIP

- CLIP (Contrastive Language-Image Pre-training)
  - Why is CLIP Special?
    - Uses **natural language** as the supervisory signal.
    - Learns from **400 million image–text pairs** collected from the Internet.
    - Enables **zero-shot recognition** using text prompts.
    - Learns more transferable representations than fixed-label supervision.

Radford et al., PMLR 2021, Learning Transferable Visual Models From Natural Language Supervision

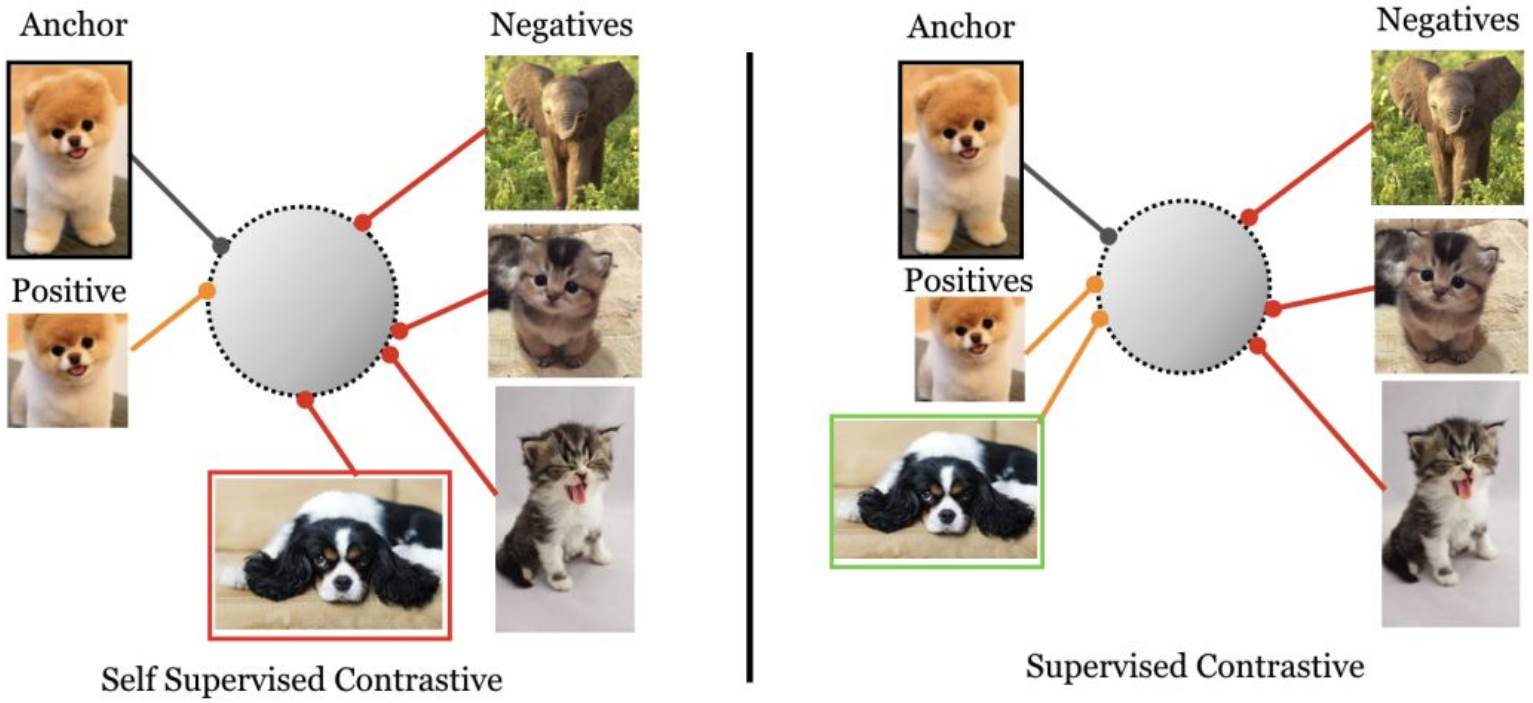
Supervised Contrastive Learning

- Supervised Contrastive Learning (SupCon)
  - Extends contrastive learning to **fully supervised settings**.
  - Treats all samples from **the same class as positive pairs.**
  - **Pulls** embeddings of the **same** class together.
  - **Pushes** embeddings from **different** classes apart.
- **Key Idea**: Use class labels to learn more discriminative representations than cross-entropy.

Khosla et al., NeurIPS 2020, Supervised Contrastive Learning

Supervised Contrastive Learning

- Supervised Contrastive Learning (SupCon)



Khosla et al., NeurIPS 2020, Supervised Contrastive Learning

Supervised Contrastive Learning

- Supervised Contrastive Learning (SupCon)

Given a set of randomly sampled n (image, label) pairs, $\{\mathbf{x}_i, y_i\}_{i=1}^{n}$, $2n$ training pairs can be created by applying two random augmentations of every sample, $\{\tilde{\mathbf{x}}_i, \tilde{y}_i\}_{i=1}^{2n}$.

Supervised contrastive loss $\mathcal{L}_{\text{supcon}}$ utilizes multiple positive and negative samples, very similar to soft nearest-neighbor loss:

Positive similarity

$$\mathcal{L}_{\text{supcon}} = -\sum_{i=1}^{2n} \frac{1}{2|N_i| - 1} \sum_{j \in N(y_i), j \neq i} \log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_j / \tau)}{\sum_{k \in I, k \neq i} \exp(\mathbf{z}_i \cdot \mathbf{z}_k / \tau)}$$

A set of Positives of the same class as Anchor

For each positive

A method that uses SimCLR's InfoNCE as is, but instead of using only one positive, uses all samples of the same class as positive, calculates each InfoNCE, and then takes the average.

Khosla et al., NeurIPS 2020, Supervised Contrastive Learning

Supervised Contrastive Learning

- Supervised Contrastive Learning (SupCon)

Dataset	SimCLR[3]	Cross-Entropy	Max-Margin [32]	SupCon
CIFAR10	93.6	95.0	92.4	**96.0**
CIFAR100	70.7	75.3	70.5	**76.5**
ImageNet	70.2	78.2	78.0	**78.7**

Table 2: Top-1 classification accuracy on ResNet-50 [17] for various datasets. We compare cross-entropy training, unsupervised representation learning (SimCLR [3]), max-margin classifiers [32] and SupCon (ours). We re-implemented and tuned hyperparameters for all baseline numbers except margin classifiers where we report published results. Note that the CIFAR-10 and CIFAR-100 results are from our PyTorch implementation and ImageNet from our TensorFlow implementation.

Experimental Results

- SupCon consistently outperforms SimCLR and Cross-Entropy.
- Improves classification accuracy across CIFAR-10, CIFAR-100, and ImageNet.
- Learns more discriminative representations than conventional supervised learning.

Key Takeaway: Contrastive objectives remain beneficial even with class labels.

Khosla et al., NeurIPS 2020, Supervised Contrastive Learning

Supervised Contrastive Learning

- Supervised Contrastive Learning (SupCon)
  - does outperform the base cross entropy, but only by a small amount.
  - outperforms the cross entropy on robustness benchmark (ImageNet-C, which applies common naturally occuring perturbations such as noise, blur and contrast changes to the ImageNet dataset).
  - is less sensitive to hyperparameter changes.

Khosla et al., NeurIPS 2020, Supervised Contrastive Learning

Can the memory bank provide more than negatives?

This ICCV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

All4One: Symbiotic Neighbour Contrastive Learning via Self-Attention and Redundancy Reduction

Imanol G. Estepa
Universitat de Barcelona,
Barcelona, Spain
igonzaes42@alumnes.ub.edu

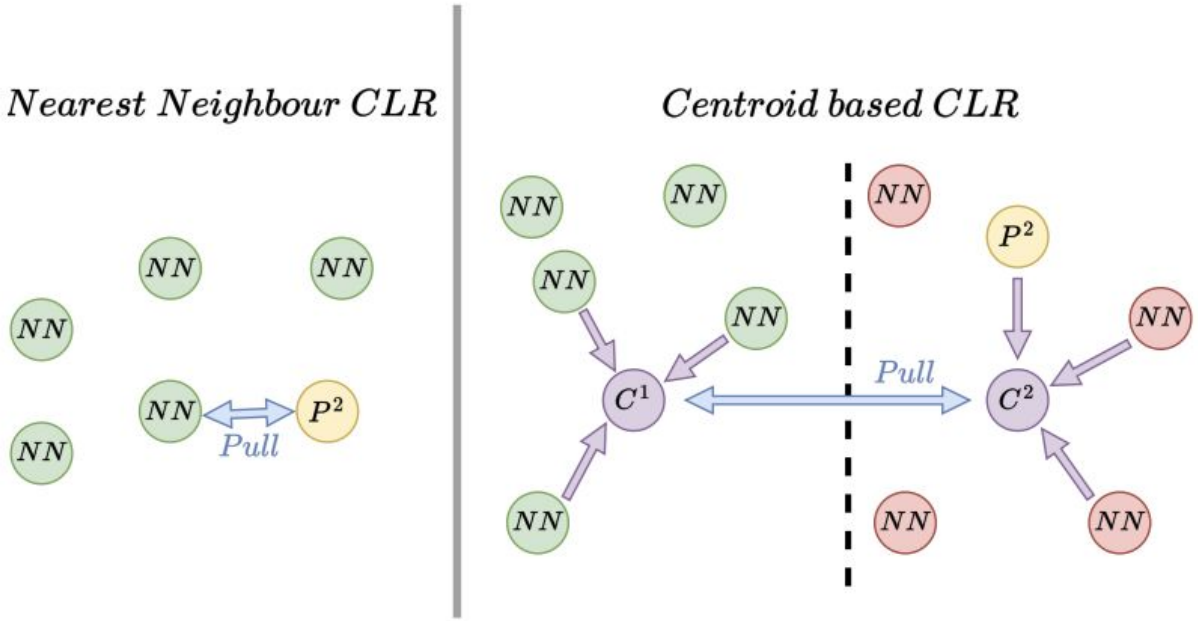
Ignacio Sarasúa
NVIDIA
isarasua@nvidia.com

Bhalaji Nagarajan
Universitat de Barcelona,
Barcelona, Spain
bhalaji.nagarajan@ub.edu

Petia Radeva
Universitat de Barcelona,
Barcelona, Spain
Computer Vision Center,
Cerdanyola (Barcelona), Spain
petia.ivanova@ub.edu

Estepa et al., ICCV 2023, All4One: Symbiotic Neighbour Contrastive Learning via Self-Attention and Redundancy Reduction

Positive Samples from a Memory Bank

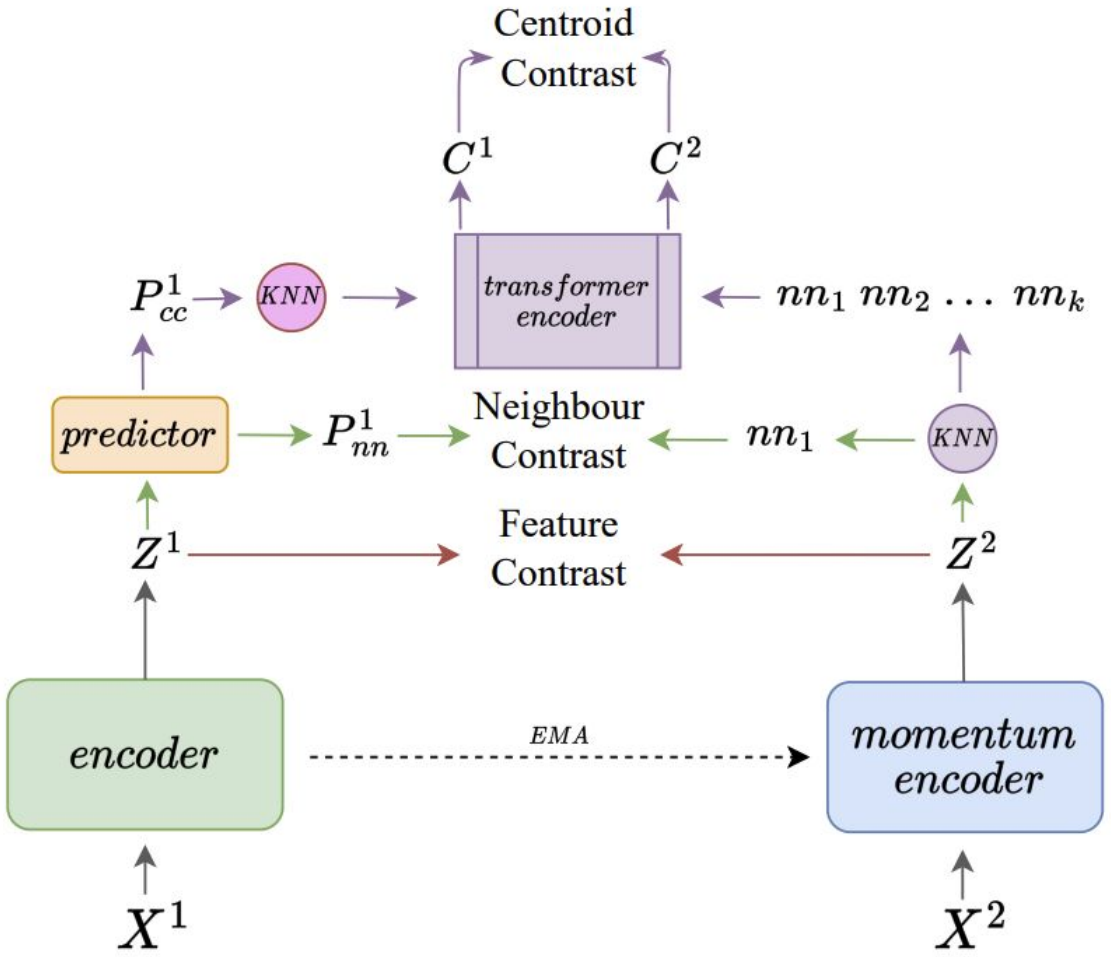


NNCLR
- Store representations in a support set.
- Retrieve the nearest neighbor.
- Use one neighbor as the positive sample.

All4One
- Retrieve K nearest neighbors.
- Combine their information.
- Compute the contrastive loss once.

NNCLR uses one neighbor. All4One summarizes multiple neighbors into one representation.

All4One Overview



Neighbor Contrast
- One nearest neighbor
- Direct positive alignment

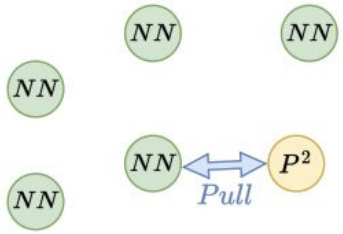
Centroid Contrast
- Multiple neighbors
- Contextual neighbor representation

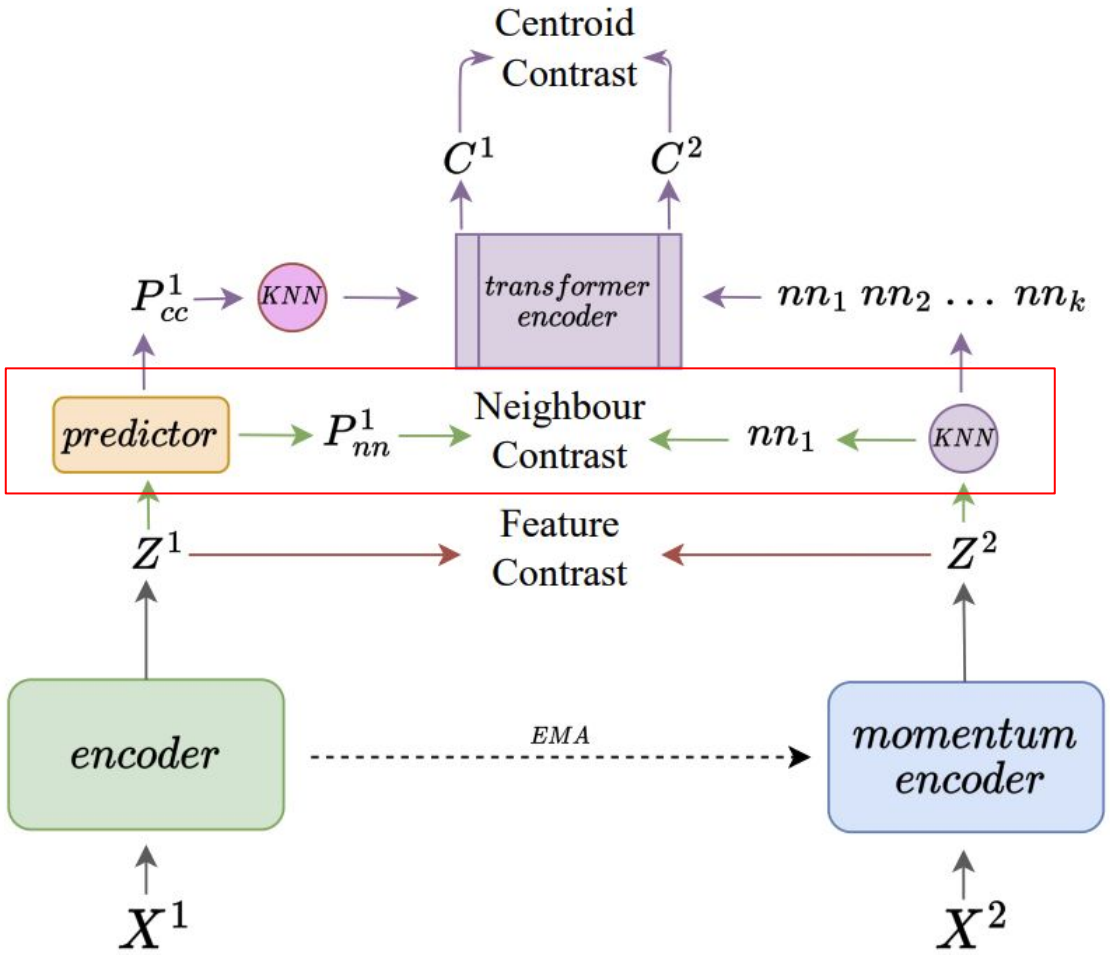
Feature Contrast
- Original augmented features
- Redundancy reduction

$$L_{All4One} = \sigma L_{NNCLR} + \kappa L_{Centroid} + \eta L_{Red}$$

All4One combines three losses defined on three different representations.

Neighbor Contrast





nearest neighbor found in the support set

online prediction from the second view

$$L_i^{NNCLR} = -log(\frac{exp(nn_i^1 \cdot p_i^2/\tau)}{\sum_{k=1}^{N} exp(nn_i^1 \cdot p_k^2/\tau)})$$

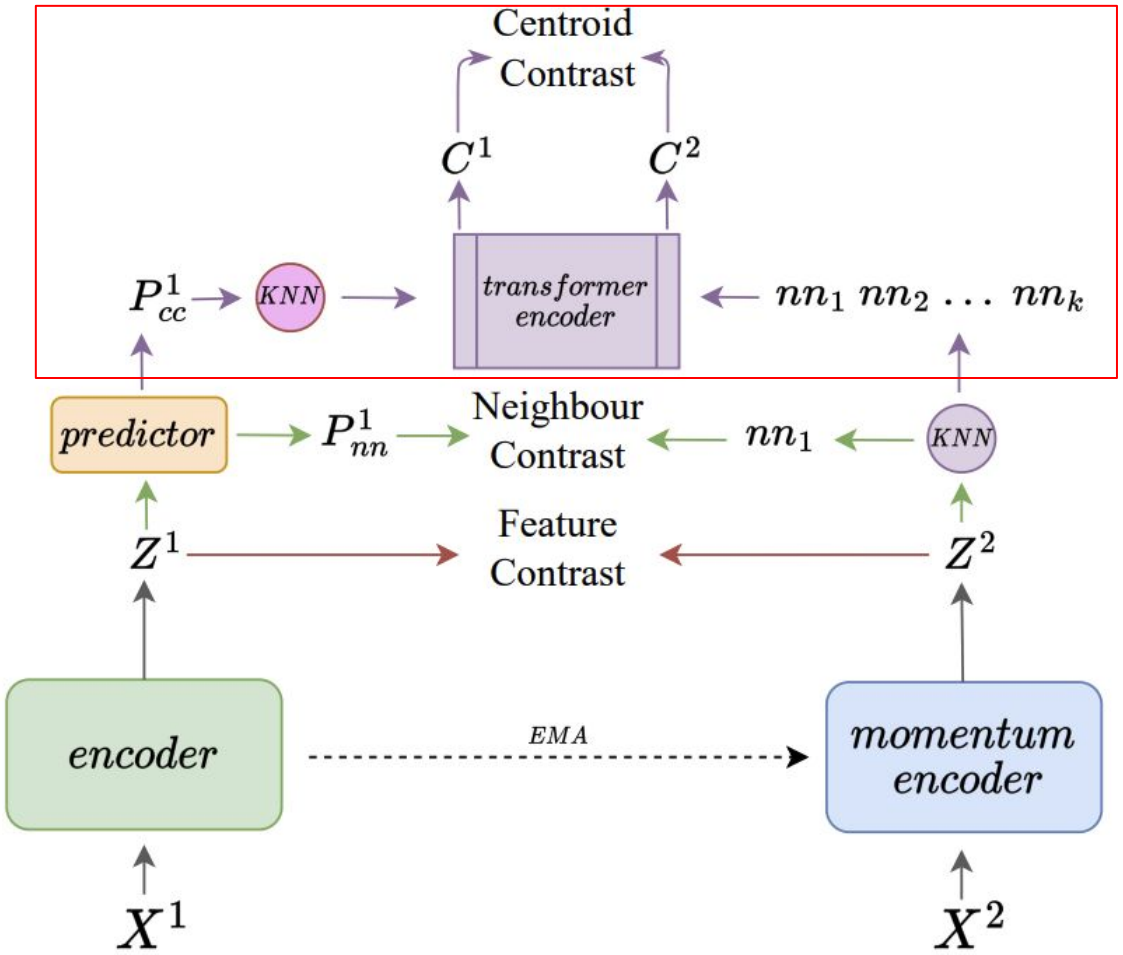
another prediction in the batch

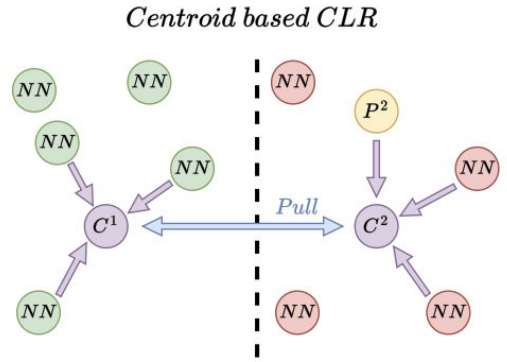
NNCLR does not directly compare two views generated from the same image

- instead, it retrieves the representation closest to the first view from a support set
- uses that neighbor as the positive sample for the second view.

The nearest neighbor replaces one augmented view in the positive pair.

Centroid Contrast





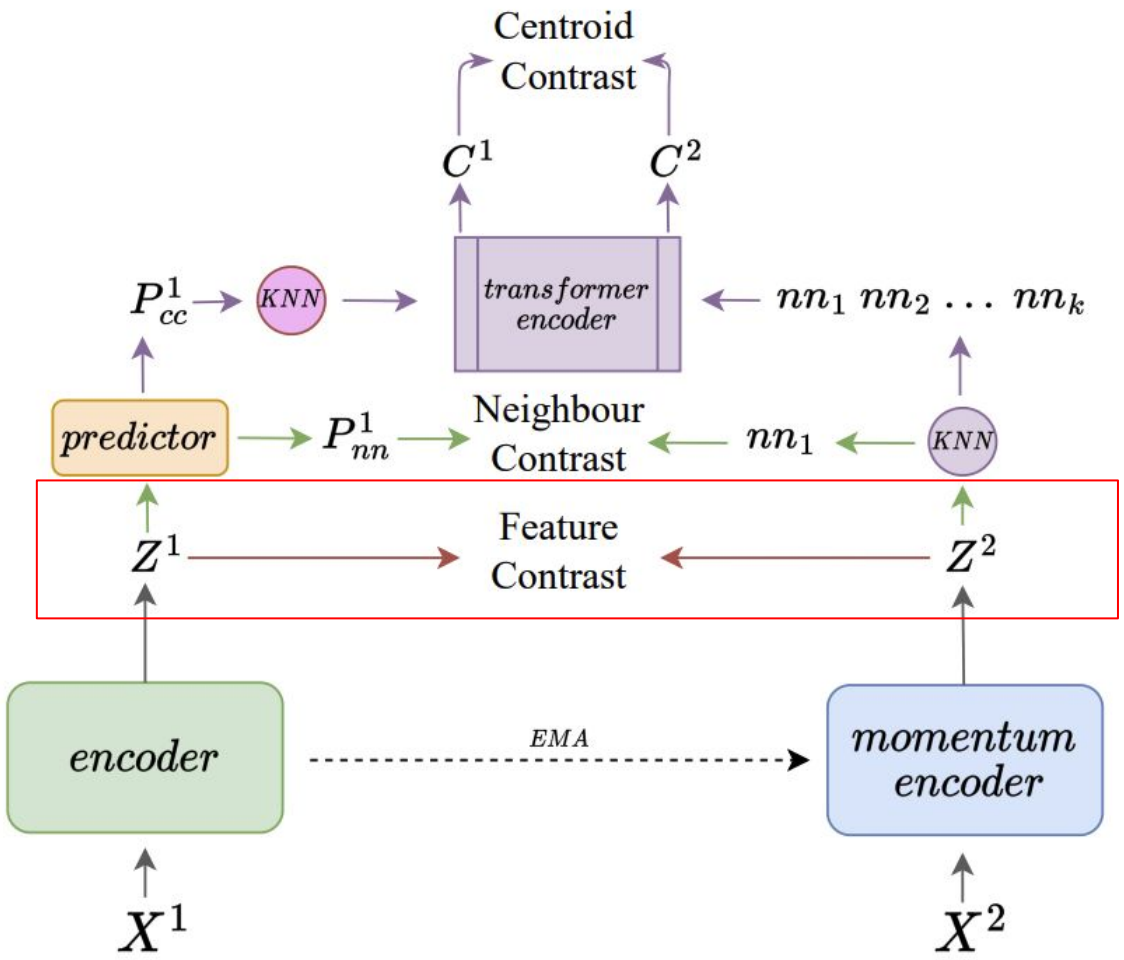
K neighbors, one contrastive comparison

$$L_i^{centroid} = -log\left(\frac{exp(c_i^1 \cdot c_i^2/\tau)}{\sum_{n=1}^{n} exp(c_i^1 \cdot c_n^2/\tau)}\right)$$

- For each query, K neighbors are retrieved.
  - When the sequence of neighbors is fed into a small Transformer encoder, information from each neighbor is integrated via self-attention.
  - The first output representation from the Transformer is used as the centroid.
- Subsequently, the centroids generated from the two views are treated as a positive pair to compute the InfoNCE loss.

The centroid is produced by self-attention. It is not an average or a cluster center.

Feature Contrast (refer Barlow Twins)



Compute cross-correlation matrix using Z
Use projected representations of the two augmented views (Z)

$$L_{Red.} = \frac{1}{2}\sqrt{\frac{1}{2D}\sum_{i=1}^{D}((1-cc^1_{ii})^2 + (1-cc^2_{ii})^2)} + \frac{1}{2}\sqrt{\frac{1}{2D(D-1)}\sum_{i=1}^{D}\sum_{j\neq i}^{D}((cc^1_{ij})^2 + (cc^2_{ij})^2)}$$

- Diagonal terms
  - The same feature dimension remains consistent across views.
- Off-diagonal terms
  - Different feature dimensions become less redundant.

Neighbor losses compare samples. Feature contrast compares feature dimensions.

Contribution of Each Objective

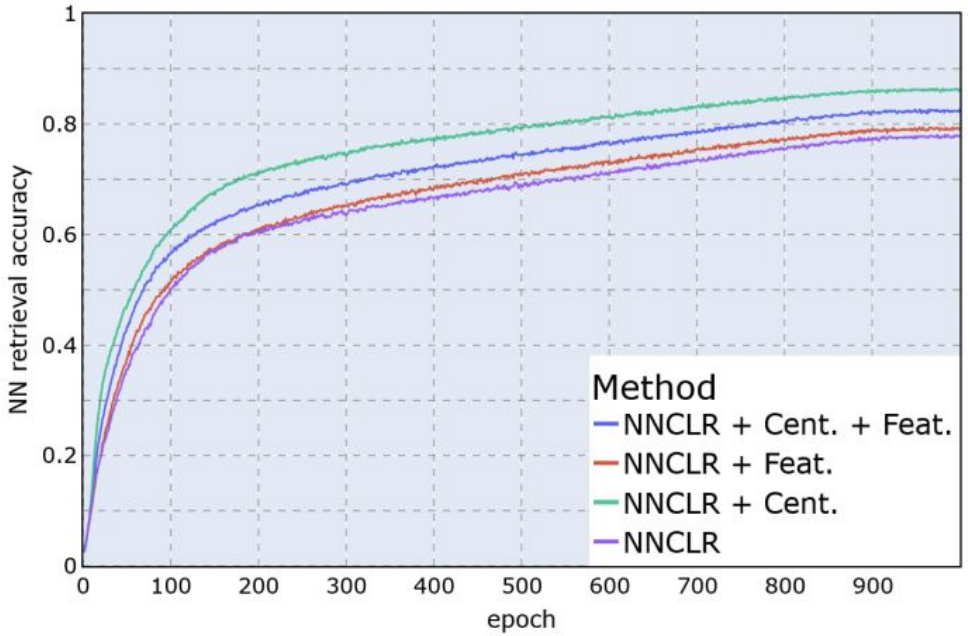
Method	NNCLR obj.	Cen. obj.	Feat. obj.	EMA	Top-1	k-NN Top-1	NN Top-1
All4One$_{v0}$	✓				69.62	62.16	68.8 (77.8*)
All4One$_{v1}$		✓			67.4	59.61	82.8
All4One$_{v2}$	✓	✓			71.02	63.21	85.28
All4One$_{v3}$	✓	✓		✓	71.08	63.83	**86.16**
All4One$_{v4}$		✓	✓	✓	71.31	63.72	80.6
All4One$_{v5}$	✓		✓	✓	71.64	64.58	78.8
All4One$_{v6}$	✓	✓	✓	✓	**72.17**	**64.84**	**82.16**

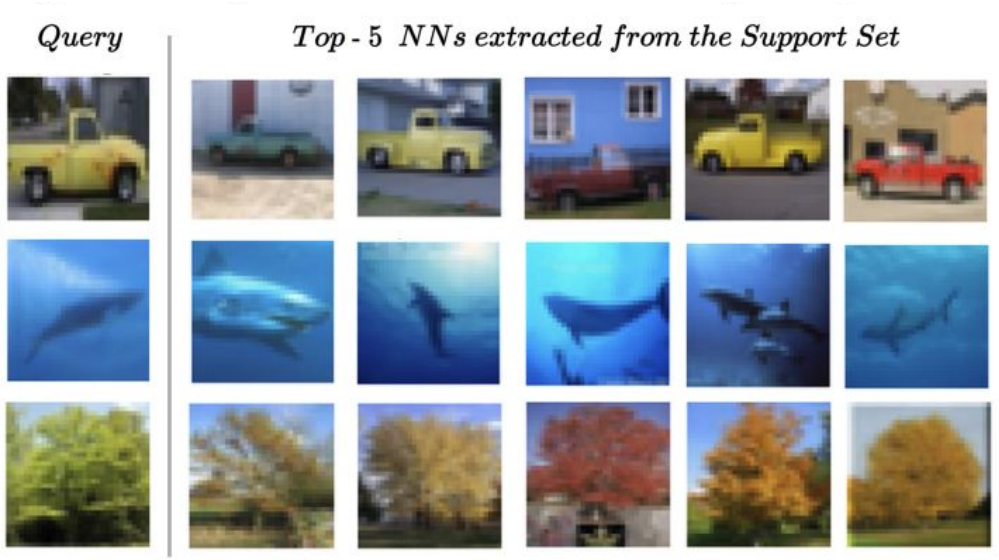
Table 3: **All4One objective function ablation study (using CIFAR-100).** All4One$_{v0}$ is equal to vanilla NNCLR. NN retrieval accuracy marked by * represents the NN retrieval obtained by increasing the queue size from 65503 to 98304 [14]

- Centroid alone: 67.4
  - Centroid contrast alone is weaker than NNCLR.
- Neighbor + Centroid: 69.62 --> 71.02
  - Multiple-neighbor context improves NNCLR.
- Full objective: 72.17
  - The three objectives work best together.

Centroid contrast is useful as a complementary objective, not as a replacement for neighbor contrast.

Neighbor Retrieval Analysi





- NN Retrieval Accuracy
  - NNCLR provides the baseline.
  - Adding the centroid objective improves retrieval.
  - The full objective does not have the highest retrieval accuracy.
  - The full objective still gives the best linear evaluation result.
- Retrieved Neighbors
  - Most retrieved samples share visual or semantic characteristics.
  - Some retrieved samples belong to different classes.
  - Different-class neighbors can still contain useful visual similarity.

Class-correct neighbor retrieval is helpful, but it is not the final objective.

Linear Evaluation

Method	CIFAR-10		CIFAR100		ImageNet100	
	Top-1	Top-5	Top-1	Top-5	Top-1	Top-5
BYOL	92.58	99.79	70.46	91.96	80.16	95.02
DC V2	88.85	99.58	63.61	88.09	75.36	93.22
DINO	89.52	99.71	66.76	90.34	74.84	92.92
MoCoV2+	92.94	99.79	69.89	91.65	78.20	95.50
MoCoV3	93.10	99.80	68.83	90.57	80.36	95.18
ReSSL	90.63	99.62	65.92	89.73	76.92	94.20
SimCLR	90.74	99.75	65.78	89.04	77.64	94.06
Simsiam	90.51	99.72	66.04	89.62	74.54	93.16
SwAV	89.17	99.68	64.88	88.78	74.04	92.70
VIbCReg	91.18	99.74	67.37	90.07	79.86	94.98
VICReg	92.07	99.74	68.54	90.83	79.22	95.06
W-MSE	88.67	99.68	61.33	87.26	67.60	90.94
BT	92.10	99.73	70.90	91.91	80.38	95.28
NNCLR	91.88	99.78	69.62	91.52	79.80	95.28
All4One	**93.24**	**99.88**	**72.17**	**93.35**	**81.93**	**96.23**

Table 1: **Linear evaluation results on CIFAR-10, CIFAR-100 and ImageNet100.** The results are extracted from Solo-learn Self-supervised learning library [11].

All4One improves both neighbor contrast and redundancy reduction baselines.

- It shows improvements across all three datasets compared to NNCLR.
- It also shows improvements compared to Barlow Twins.
- The margin of improvement is greatest on CIFAR-100.
- This suggests that multiple-neighbor context is likely more useful for datasets containing many similar classes.

How Many Neighbors?

- A small Transformer is sufficient.
- Five neighbors give the best result.
- More neighbors introduce additional computation.
- Less relevant neighbors can dilute the contextual representation.

Layer number	Top-1	k-NN Top-1
3	72.17	64.84
6	71.86	64.50
9	71.75	64.52

(c) **Number of transformer layers.**

Number of NN	Top-1	k-NN Top-1
5	72.17	64.84
10	72.00	64.54
15	71.92	64.63
20	71.79	64.6

(d) **Number of NNs extracted.**

- If the Transformer is too powerful, it might take over the role of generating representations from the backbone.
  - Since the goal of the paper is to train a backbone for downstream tasks, a smaller Transformer is more appropriate.

- Having a large number of neighbors is not always beneficial;
  - as the number of neighbors increases, samples with low relevance may be included, potentially diluting the contextual information.

Can cluster-based SSL preserve fine-grained visual details?

This ICCV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Prototype-based Contrastive Learning with Stage-wise Progressive Augmentation for Self-Supervised Fine-Grained Learning

Baofeng Tan[1] Xiu-Shen Wei[2*] Lin Zhao[1]

[1]School of Computer Science and Engineering, Nanjing University of Science and Technology
[2]School of Computer Science and Engineering, and Key Laboratory of New Generation Artificial Intelligence Technology and Its Interdisciplinary Applications, Southeast University

Tan et al., ICCV 2025, Prototype-based Contrastive Learning with Stage-wise Progressive Augmentation for Self-Supervised Fine-Grained Learning

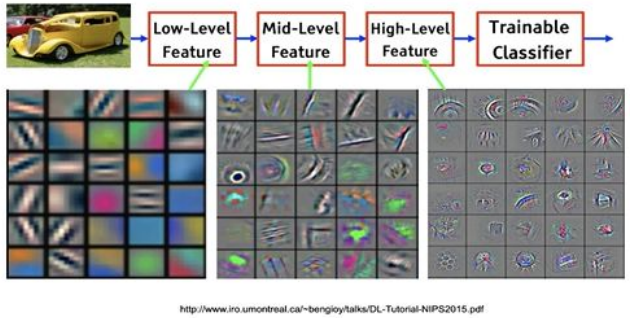
Fine-Grained Recognition Needs Fine Details

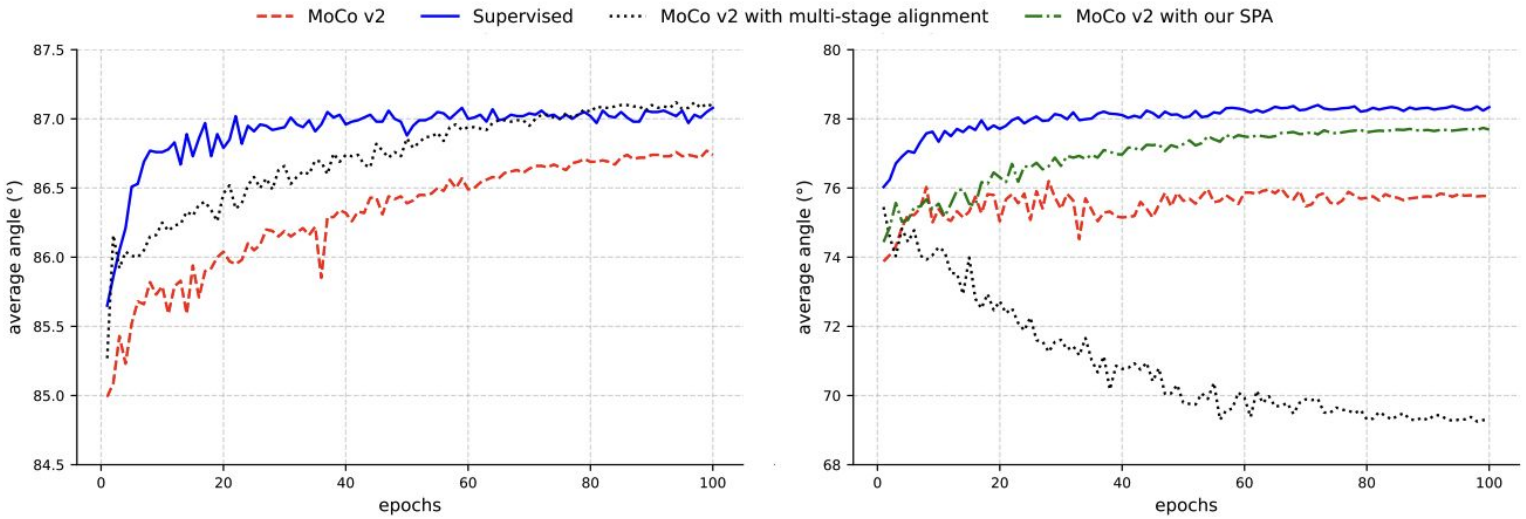
- Generic recognition
  - “Is this a bird?”
  - Global shape may be sufficient.
- Fine-grained recognition
  - “Which bird species is this?”
  - Small parts become important.
  - Beak, head, wing, and tail patterns may distinguish classes.

Fine-grained recognition requires both holistic and part-level representations.

Why Does Multi-Stage Alignment Fail?

- Multi-stage alignment
  - Align representations at several backbone stages.
  - Helpful on a generic dataset (Caltech).
  - Harmful on a fine-grained dataset (CUB).





(a) Training on *Caltech-256*.

(b) Training on *CUB200*.

The vertical axis: the average angle between different image representations.

- Larger angle → Different samples are better separated.
- Smaller angle → Representations are clustered closer together.

Why does the same alignment strategy fail on fine-grained data?

Strong Augmentation Changes Shallow Features

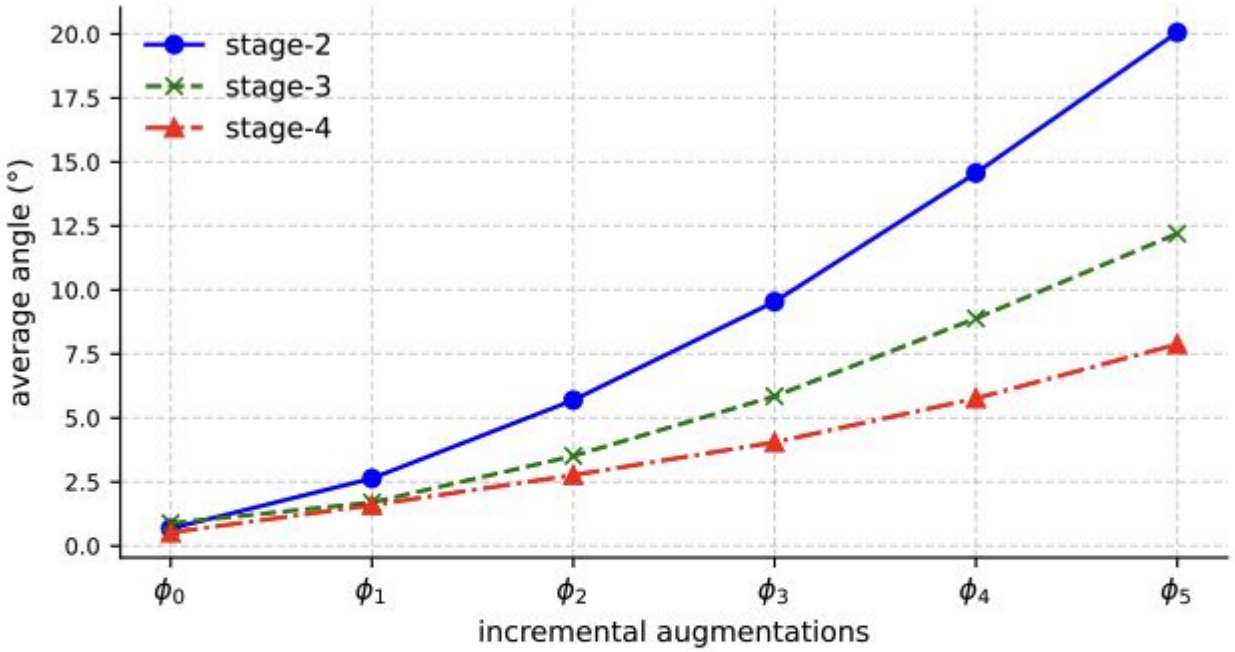


Figure 2. The average angle between embeddings and their positive views, generated by different stages of f_q and f_k. The horizontal axis represents a set of incremental augmentations defined in Section 3. As the intensity of augmentations increases, the average angles at different stages steadily rise.

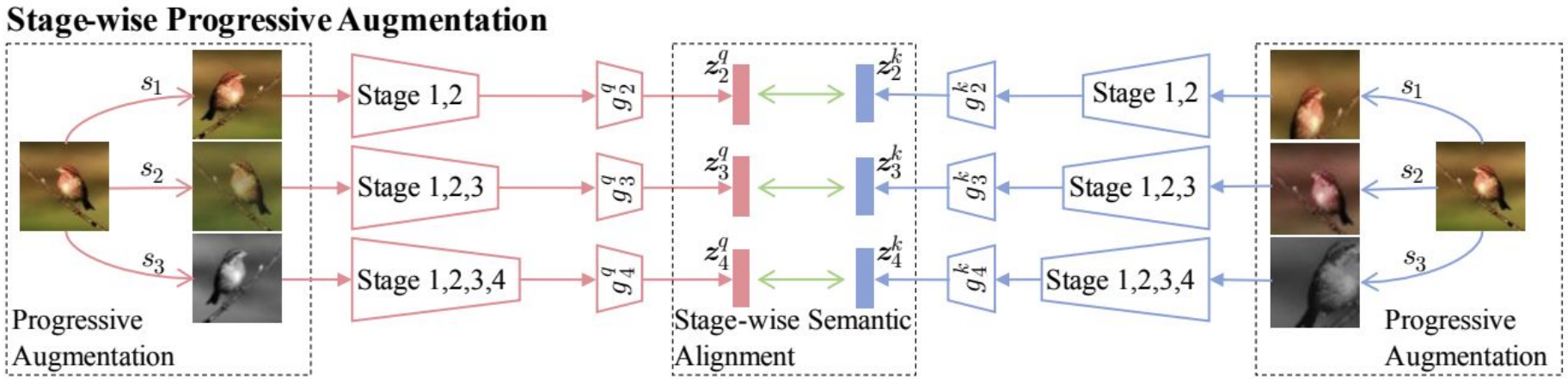
- Stronger augmentation increases positive-pair disagreement.
- Shallow stages are affected more severely.
- Deep stages are relatively augmentation-tolerant.

When strong augmentation is applied,

- shallow layers perceive two photos of the same bird as different entities.
- In contrast, deep layers are relatively less affected because they focus on the overall semantic meaning.

→ One augmentation strength does not fit every stage.

PAPN: Two Ideas for Fine-Grained SSL



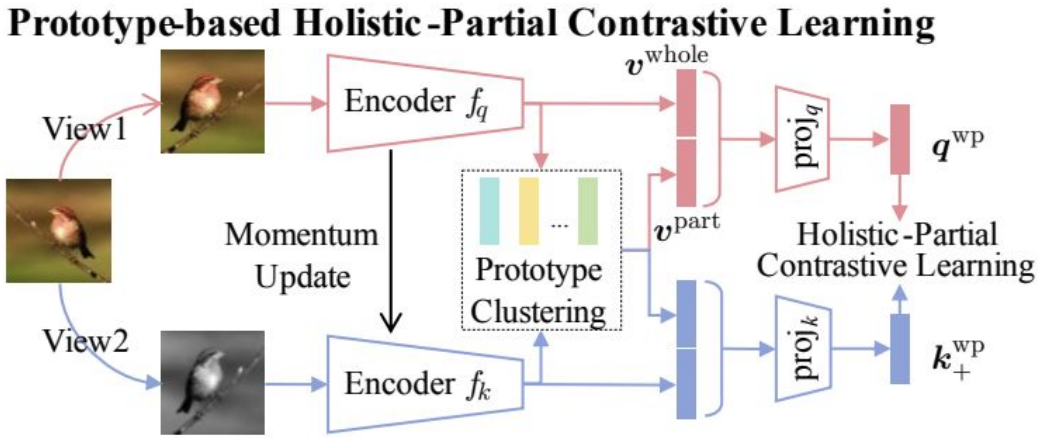
① Stage-wise Progressive Augmentation
- Mild augmentation for shallow stages
- Strong augmentation for deep stages
- Stage-wise semantic alignment

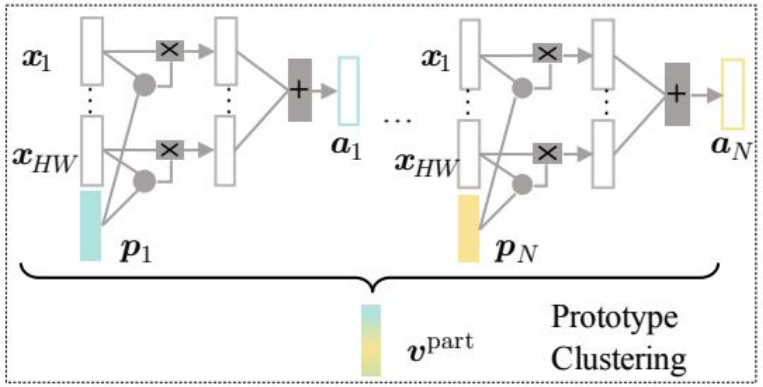
$$\mathcal{L}_{SPA} = \sum_{i=1}^{T} ||\hat{z}^k_{L-T+i} - \hat{z}^q_{L-T+i}||_2^2$$

where $\hat{z} = z/||z||_2$.

Positive pairs are created based on the intensity of augmentation that each stage can withstand.

PAPN: Two Ideas for Fine-Grained SSL





② Prototype-based Holistic-Partial Learning

- Learn prototypes from spatial descriptors
  - View the final feature map as a set of spatial descriptors.
- Extract part-level representations
- Combine global and local information

$$\boldsymbol{X}_L = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_{HW}]$$

$$\boldsymbol{P} = [\boldsymbol{p}_1, \boldsymbol{p}_2, \ldots, \boldsymbol{p}_N]$$

$$\boldsymbol{M} = \hat{\boldsymbol{P}}^\top \hat{\boldsymbol{X}}_L$$

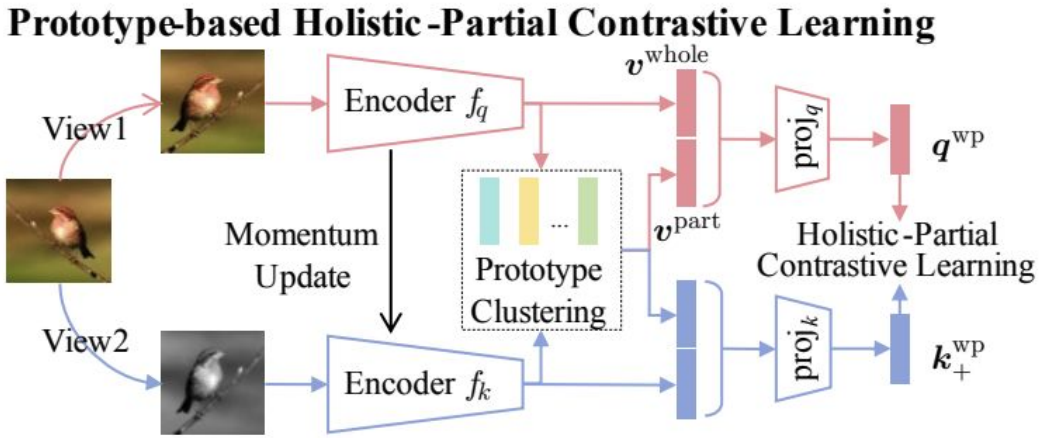
Prototype similarity

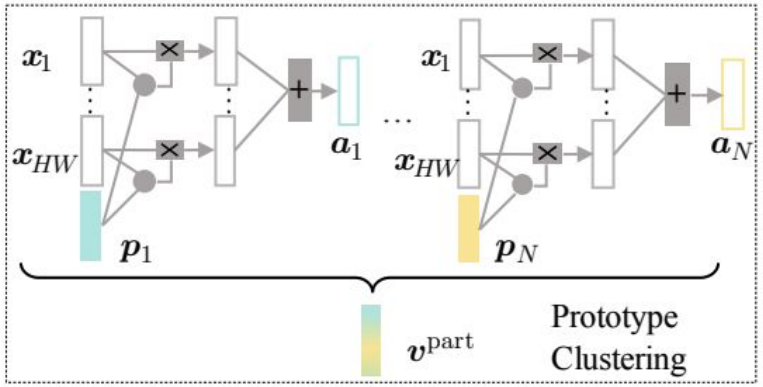
Part representation

$$\boldsymbol{A} = \boldsymbol{X}_L \boldsymbol{M}^\top = [\boldsymbol{a}_1, \boldsymbol{a}_2, \ldots, \boldsymbol{a}_N]$$

$$\boldsymbol{v}^{\text{part}} = \frac{\sum_{i=1}^{N} \boldsymbol{a}_i}{N}$$

PAPN: Two Ideas for Fine-Grained SSL





$$v^{\text{whole}} = [\text{pool}(X_{L-T+1}); \ldots; \text{pool}(X_L)],$$
$$v^{\text{wp}} = [v^{\text{whole}}; v^{\text{part}}].$$

- Whole feature: overall object appearance
- Part feature: local discriminative patterns
- Concatenated representation enters InfoNCE

$$\mathcal{L}_{PHP} = -\log \frac{\exp(q^{\text{wp}} \cdot k_+^{\text{wp}}/\tau)}{\sum_{i=0}^{K} \exp(q^{\text{wp}} \cdot k_i^{\text{wp}}/\tau)},$$

Do not choose between global and local features—contrast both together.

Do Both Components Help?

Table 3. Top-1 classification accuracy (in %) and rank-1, rank5 retrieval accuracy (in %) of different configs in ablation studies.

Configs	SPA	PHP	*CUB200*		
			top-1	rank-1	rank-5
#1			63.98	39.72	67.14
#2	✓		67.65	43.77	71.34
#3		✓	67.91	43.31	70.70
#4	✓	✓	**69.93**	**45.39**	**72.81**

- SPA only: +3.67 top-1
- PHP only: +3.93 top-1
- SPA + PHP: +5.95 top-1

Do Both Components Help?

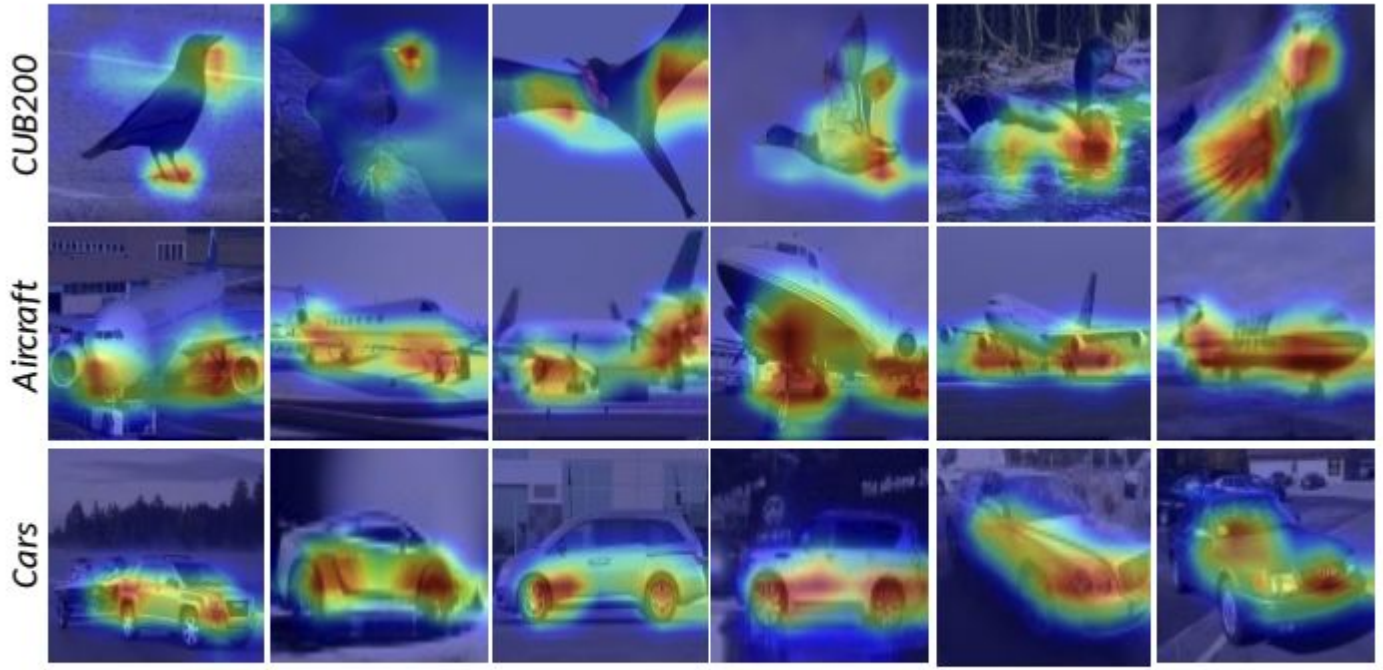


Figure 5. Visualization of part localizations on three datasets.

- The heatmaps obtained from the prototype show responses in the following areas:
  - Bird: head, wing, tail
  - Aircraft: engine, wing, tail
  - Car: wheel, hood, body

Progressive augmentation preserves semantics; prototypes expose discriminative parts.

Main Results and Takeaways

- PAPN improves fine-grained representations by:
  - 1. Matching augmentation strength to feature depth.
  - 2. Discovering local patterns with learnable prototypes.
  - 3. Combining whole- and part-level features in InfoNCE.

Table 1. Performance comparison of various methods on classification and retrieval tasks. The results include top-1 classification accuracy (in %), as well as rank-1 and rank-5 retrieval accuracies (in %). All results of baselines except DINOv2 are reported in [38, 43]. ResNet-50†: Indicates the use of ImageNet-1k pretraining weights, where the backbone is frozen and only the linear classifier is fine-tuned. DINOv2*: Results reproduced using the official implementation adapted for ViT-S, with weights pre-trained on ImageNet-1k for a fair comparison.

Methods	Classification			Retrieval					
	CUB200	*Cars*	*Aircraft*	*CUB200*		*Cars*		*Aircraft*	
				rank-1	rank-5	rank-1	rank-5	rank-1	rank-5
ResNet-50†	63.06	61.41	49.79	40.39	68.94	29.28	54.66	29.48	51.39
supervised	77.46	88.60	85.93	74.43	90.99	80.54	94.35	74.16	90.47
SimSiam [6]	46.75	45.72	38.52	16.24	-	12.45	-	18.49	-
MoCo v2 [7]	63.98	62.02	51.13	39.72	67.14	30.51	56.15	30.02	52.87
LEWEL [17]	64.59	62.91	51.90	39.91	-	32.36	-	31.09	-
ContrastiveCrop [29]	64.23	63.29	52.04	39.84	-	32.71	-	30.37	-
SAM-SSL-Bilinear [37]	64.94	62.85	52.83	40.08	-	33.19	-	30.52	-
BarlowTwins [52]	33.45	31.91	34.77	-	-	-	-	-	-
VICReg [3]	37.78	30.80	36.00	-	-	-	-	-	-
LCR [38]	65.24	63.96	53.22	41.26	-	34.74	-	31.55	-
DINOv2* [28]	66.02	64.47	54.67	41.65	70.28	34.01	57.12	32.79	57.64
LDF [43]	66.17	65.60	55.28	42.06	69.59	35.81	**61.94**	33.27	56.80
Our PAPN	**69.93**	**67.48**	**60.13**	**45.39**	**72.81**	**35.98**	59.94	**35.13**	**58.75**

From clustering images to discovering reusable visual parts.

What if a mini-batch contains Imprecise Class Labels? (in supervised contrastive learning)

Understanding and Mitigating Human-Labelling Errors in Supervised Contrastive Learning

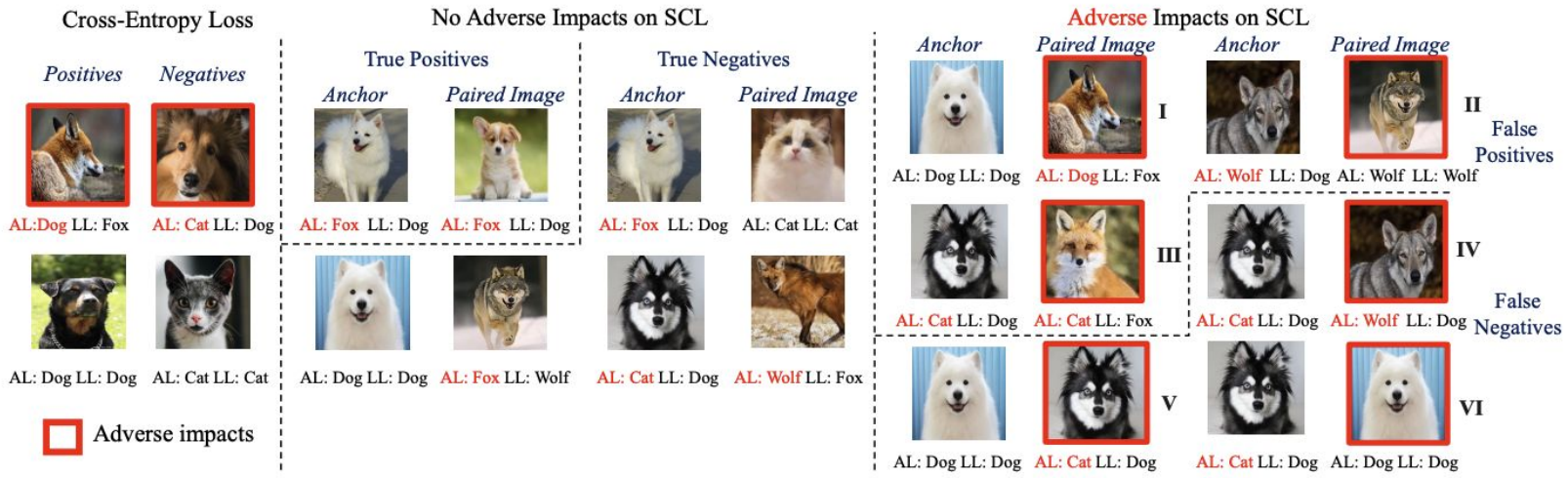
Zijun Long[1], Lipeng Zhuang[1], George Killick[1], Richard McCreadie[1], Gerardo Aragon-Camarasa[1], and Paul Henderson[1]

University of Glasgow, United Kingdom

Long et al., ECCV 2024, Understanding and Mitigating Human-Labelling Errors in Supervised Contrastive Learning

Human Label Errors in SupCon

- It demonstrates that label errors affect Cross-Entropy (CE) and Supervised Contrastive Learning (SCL) differently.
  - In Cross-Entropy, an incorrect label results in an incorrect learning signal for that sample.
  - SCL learns based on the relationship between two samples.
  - Even if some labels are incorrect, the pairwise relationship can still be correct.
  - False positives or false negatives occur depending on the relationship between the assigned label and the latent label.



Assigned label (AL)
- Label provided by an annotator

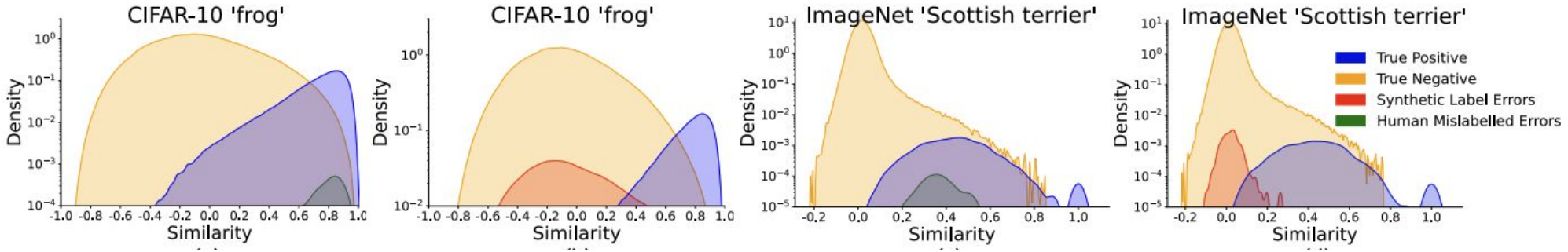
Latent label (LL)
- True semantic class of the image

A mislabeled sample does not always produce an incorrect contrastive pair.

The incorrect pair relationships resulting from label errors, rather than the label errors themselves, directly impact SCL.

Human Errors Differ from Synthetic Errors

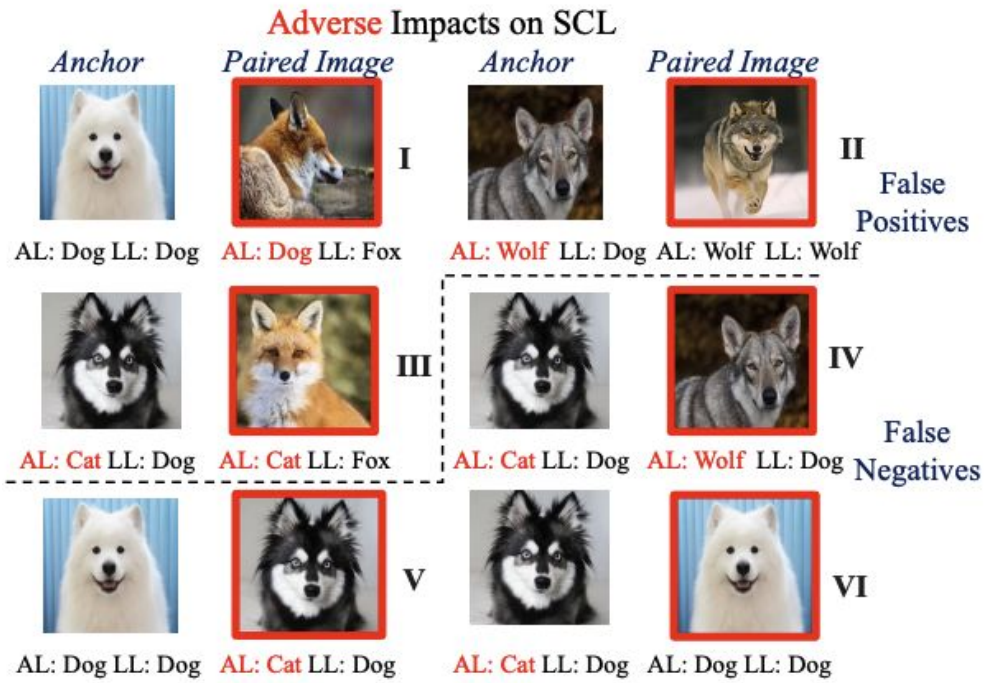
- Synthetic label errors
  - Labels are changed randomly.
  - Mislabeled samples often look different from the assigned class.
- Human label errors
  - Humans confuse visually similar classes.
  - Mislabeled samples often remain close to the assigned class.



Human labeling errors overlap strongly with true positives.

False Positives Dominate the Errors

- False positive
  - Assigned labels are the same
  - Latent labels are different
  - The model incorrectly pulls the two representations together.
- False negative
  - Assigned labels are different
  - Latent labels are the same
  - The model incorrectly pushes out both representations.
- If we let τ be the label error rate and C be the number of classes



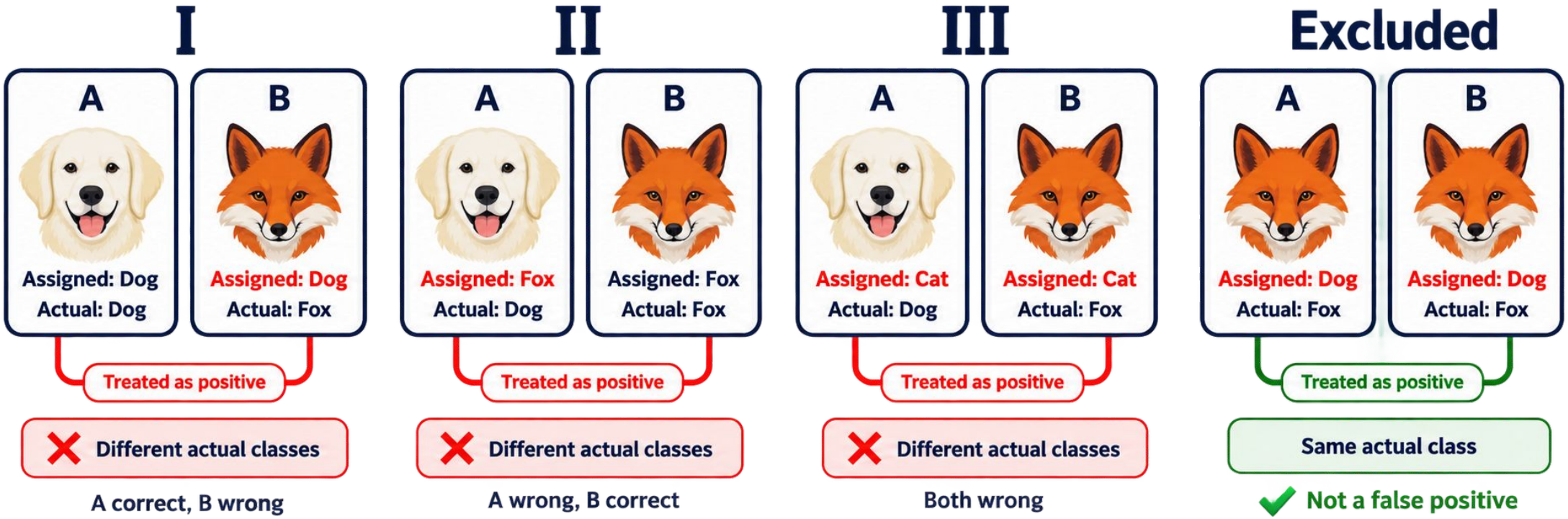
$$P_{FN} = \frac{\tau^2}{(C-2)^2} + \frac{2\tau - 2\tau^2}{C-1}$$

$$P_{FP} = \underbrace{2\tau - \tau^2}_{\text{at least one error}} - \underbrace{\frac{\tau^2}{(C-1)^2}}_{\text{both wrong but same latent class}}$$

$$P_{FN} \approx \frac{2\tau}{C-1} \qquad P_{FP} \approx 2\tau$$

With many classes, almost all incorrect SCL signals come from false positives.

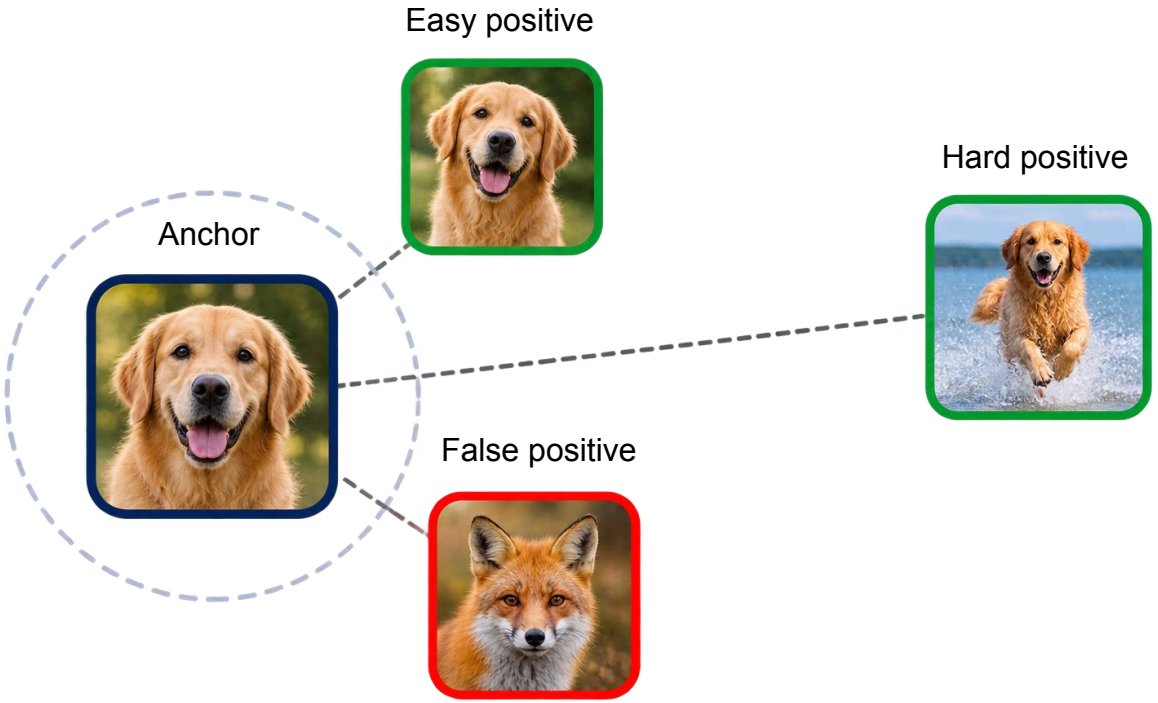
False Positives Dominate the Errors



At least one wrong: $2\tau - \tau^2$ − Both wrong but same actual class: $\frac{\tau^2}{(C-1)^2}$ = P_{FP}

(1-τ)τ τ(1-τ) ττ

The Problem with Easy Positives



Problem

- Human labeling errors often satisfy the following two conditions simultaneously:
  - Same assigned label
  - High feature similarity

→ Consequently, the model may classify them as high-confidence positives.

Risks associated with existing noise mitigation methods:

- Confident pairs are assigned higher weights.
- Human labeling errors can appear highly confident.
- The model reinforces false-positive relationships.

- Generally, "positives" located nearby are considered reliable. However, human labeling errors involve samples that humans confused due to visual similarity; consequently, they may already be close to the anchor in the representation space.

High similarity does not always imply a correct positive pair.

SCL-RHE: Deprioritizing Easy Positives

- Core Idea: High similarity → Lower sampling weight
  - Easy positives' that are already close to the anchor provide little new information for representation learning; moreover, they may include instances of human labeling error.
  - SCL-RHE reduces the weight assigned to these examples and places greater emphasis on 'hard positives' → samples belonging to the same class that are currently distant from the anchor.

$$q^+(x^+) := q\left(x^+ \mid z(x) = z(x^+)\right) \propto \frac{1}{e^{\beta f(x)^T f(x^+)}} \cdot p_x^+(x^+)$$

The positive sampling distribution to be used by SCL-RHE

The condition that the anchor and the positive candidate share the same actual class.

Similarity between anchor and positive

true positive distribution

Positive type	Similarity	Weight
Easy positive	High	Low
Hard positive	Low	High

Most Incorrect Signals Are False Positives

- CIFAR-100
  - 99.04% of incorrect contrastive signals come from false positives.
  - 0.96% come from false negatives.
- ImageNet-1K
  - 99.91% of incorrect contrastive signals come from false positives.
  - 0.09% come from false negatives.
- Correcting false positives addresses almost all error signals.
- Negative correction provides only a small additional benefit.
- SCL-RHE focuses primarily on positive sampling.

Positive pairs are the main source of label-error propagation in SCL.

Results from Training from Scratch

- SCL-RHE consistently improves over SupCon.
- Improvement is larger for BEiT-3.
- Methods designed for heavy synthetic noise can underperform standard SupCon.

Table 1: Model accuracy measured using the acc@1 metric when trained with different loss functions on 3 popular image classification benchmarks. All models here are trained from scratch using only the indicated dataset, without pre-training.

Model	Loss	CIFAR-10	CIFAR-100	ImageNet-1K
BEiT-3	CE [43]	71.70	59.67	77.91
	SupCon [27]	88.96	60.77	82.57
	Sel-CL [29]	86.33	59.51	81.87
	TCL [24]	85.16	59.22	81.74
	Ours	**90.16**	**64.47**	**84.21**
ResNet-50	CE [43]	95.00	75.30	78.20
	SupCon [27]	96.00	76.50	78.70
	Sel-CL [29]	93.10	74.29	77.85
	TCL [24]	92.80	74.14	77.17
	Ours	**96.39**	**77.82**	**79.15**

Sel-CL and TCL are designed to aggressively remove synthetic noise. However, since the actual human noise rate is relatively low, removing too many samples ends up reducing the training data.

Corrected Labels Reveal a Larger Gain

Table 2: Accuracy of the BEiT-3 model using the metric acc@1 on different datasets and with various loss functions, when evaluating original and corrected test-set labels.

Loss	Test set	CIFAR-10	CIFAR-100	ImageNet-1K
CE [43]	Original	71.70	59.67	77.91
	Corrected	71.79 (+0.09)	59.82 (+0.15)	78.24 (+0.33)
SupCon [27]	Original	88.96	60.77	82.57
	Corrected	89.11 (+0.15)	61.49 (+0.72)	83.74 (+1.17)
Sel-CL [29]	Original	86.33	59.51	81.87
	Corrected	86.21 (-0.12)	58.62 (-0.89)	81.35 (-0.52)
TCL [24]	Original	85.16	59.22	81.74
	Corrected	84.97 (-0.19)	58.14 (-1.08)	81.28 (-0.46)
Ours	Original	90.16	64.47	84.21
	Corrected	**90.41** (+0.25)	**65.34** (+0.87)	**86.02** (+1.81)

- Since the original test labels contain errors, the actual performance improvement may be underestimated.
- The extent of improvement for SCL-RHE appears greater on the corrected test set.

Table 4: Classification accuracy after fine-tuning a pre-trained BEiT-3 with different loss functions, on several benchmarks.

Model	Loss	CIFAR-100	CUB-200	Caltech-256	Oxf-Flowers	Oxf-Pets	iNat2017	Places365	ImageNet-1K
ViT	CE [43]	87.13	76.93	90.92	90.86	93.81	65.26	54.06	77.91
BEiT-3	CE [43]	92.96	98.00	98.53	94.94	94.49	72.31	59.81	85.40
BEiT-3	SupCon [27]	93.15	98.23	98.66	95.10	94.52	72.85	60.31	85.47
BEiT-3	Sel-CL [29]	91.48	94.52	97.19	93.71	94.51	72.43	58.36	85.21
BEiT-3	TCL [24]	90.92	93.89	97.26	93.89	94.68	72.47	59.22	85.18
BEiT-3	Ours	**93.81**	**98.95**	**99.41**	**95.89**	**96.41**	**76.25**	**62.53**	**86.51**

In Table 4, SCL-RHE achieves the highest accuracy across all eight downstream datasets.
Notably, compared to SupCon:

- iNaturalist 2017: +3.4%p
- Places365: +2.2%p

Human labeling errors often become easy false positives.

SCL-RHE reduces their influence by emphasizing informative hard positives.

Next Week..

- Semi-supervised learning ..